

Predictive Maintenance for Server Failure in Virtual Environments

By

Wong Pei Kei

A PROPOSAL

SUBMITTED TO

Universiti Tunku Abdul Rahman

in partial fulfillment of the requirements

for the degree of

BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)

COMMUNICATIONS AND NETWORKING

Faculty of Information and Communication Technology

(Kampar Campus)

FEB 2025

ACKNOWLEDGEMENTS

I would like to take this opportunity to thank all the people who have supported me and are still supporting me throughout the journey of researching and developing this project on predictive maintenance for server failure. I am truly grateful to my supervisor, Dr. Aun Yichiet, for being highly supportive of the project's direction and for providing valuable feedback that helped improve the effectiveness of the solution in addressing real-world problems. A special thanks goes to my parents, who have been incredibly supportive of my studies and have always made sure that I am in good health, both physically and mentally. Through this journey, I have come to realize how important our well-being is, as being in poor shape can seriously affect the quality of both the research and the final report. One key factor behind the success of this project is that the topic was personally chosen by me. Because of that, I felt a strong sense of responsibility and passion, dedicating countless days and nights to ensure the outcome met my expectations and standards. I genuinely hope that this report will be helpful to anyone working on a similar topic or looking for guidance in this area. Thank you very much, and I wish you the best of luck with your own research journey.

COPYRIGHT STATEMENT

© 2025 Wong Pei Kei. All rights reserved.

This Final Year Project proposal is submitted in partial fulfillment of the requirements for the degree of **Bachelor of Information Technology (Honours) Communications and Networking** at Universiti Tunku Abdul Rahman (UTAR). This Final Year Project proposal represents the work of the author, except where due acknowledgment has been made in the text. No part of this Final Year Project proposal may be reproduced, stored, or transmitted in any form or by any means, whether electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the author or UTAR, in accordance with UTAR's Intellectual Property Policy.

ABSTRACT

This project focuses on developing a predictive maintenance system to detect potential server failures before they occur. By utilizing a variety of real-world datasets – covering different processor types, hardware metrics, failure types, and usage patterns, the system builds a robust machine learning model capable of handling diverse server environments. The workflow includes data preprocessing, feature engineering, model training, evaluation, and deployment, with techniques such as SMOTE and GridSearchCV applied to improve model performance. Real-time telemetry from HWINFO, OHM, and ZabbixAgent is used alongside simulated data to validate predictions. Although access to sensitive failure data remains a challenge, this project demonstrates a reliable solution by combining historical and real-time data, offering practical applications in failure prediction, risk classification, and maintenance planning.

Area of Study: **Predictive Maintenance, Machine Learning**

Keywords: **Server Failure Prediction, Hardware Metrics, Real-time Telemetry, Data Analysis**

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	III
COPYRIGHT STATEMENT.....	IV
ABSTRACT	V
TABLE OF CONTENTS	VI
LIST OF FIGURES	IX
LIST OF TABLES	X
LIST OF SYMBOLS	XI
LIST OF ABBREVIATIONS.....	XIII
CHAPTER 1.....	15
Introduction	15
1.1 Problem Statement and Motivation	17
1.2 Research Objectives	19
1.3 Project Scope and Direction.....	21
1.4 Contributions.....	23
1.5 Background Information	24
1.6 Report Organization	26
CHAPTER 2.....	27
Literature Reviews	27
2.1 Prior Arts in Predictive Maintenance for Server Failure.....	27

2.1.1 Predictive Maintenance in Server Infrastructure.....	27
2.1.2 Hardware Health Monitoring	28
2.1.3 Techniques for Failure Prediction	28
2.2 Datasets and Data Collection for Predictive Maintenance	30
2.2.1 Processor & Motherboard Datasets.....	30
2.2.2 Sensor & Hardware Metrics Datasets	30
2.2.3 Failure Type Datasets	31
2.2.4 Usage Pattern Datasets.....	31
2.3 Fact Finding and Data Analysis Techniques.....	32
2.3.1 Scientific Methods in PM Data Analysis	32
2.3.2 Data Element Analysis.....	33
2.3.3 Report Use Analysis	34
2.4 Existing Predictive Maintenance Models and Techniques	36
2.4.1 Machine Learning Models in PM	36
2.4.2 Deep Learning Models.....	36
2.4.3 Hybrid Models.....	37
2.5 Critical Review of Previous Works	38
2.5.1 Strengths of Existing Approaches	38
2.5.2 Weaknesses and Gaps	38
2.5.3 Comparison with Proposed Solution	39
CHAPTER 3.....	42
Proposed Method/Approach.....	42
3.1 System Requirement.....	42
3.1.1 Hardware.....	42
3.1.2 Software	43
3.1.3 Data Processing Environment	44
3.2 System Design	45
3.2.1 Architecture Development.....	45
3.2.2 System Equation.....	51
3.3 System Architecture Diagram.....	53
3.4 Implementation Issues and Challenges	54
3.5 Timeline.....	56
CHAPTER 4.....	58
Preliminary Work.....	58
4.1 Setting up.....	58

4.2 Data Exploration and Integration.....	59
4.3 Predictive Maintenance Model Development.....	60
4.4 Feasibility Analysis	65
4.5 User Case Scenario	66
4.6 Latency Measurement of Prediction Module.....	68
4.7 Scalability Consideration	69
4.8 Future Work	70
CHAPTER 5.....	71
Conclusion.....	71
REFERENCES	73
APPENDIX A	80
A.1 Code Sample	80
A.2 Poster	81

LIST OF FIGURES

Figure Number	Title	Page
Figure 2.3.1	Sensor Data Flow.	28
Figure 2.3.2	Zabbix Data Flow.	30
Figure 3.1.1	Specifications of Laptop.	38
Figure 3.1.2	Zabbix.	39
Figure 3.1.3	HWiNFO.	39
Figure 3.1.4	OHM.	40
Figure 3.1.5	Jupyter Notebook.	40
Figure 3.2.1	System Design Diagram.	44
Figure 3.2.2	Predictive Maintenance Workflow.	49
Figure 3.3	Architecture for Real-time Risk Assessment using Machine Learning.	52
Figure 3.5	Timeline.	55
Figure 4.3.1	Correlation Heatmap of Engineered Features.	59
Figure 4.3.2	Classification Report for Models on AI4I dataset.	60
Figure 4.3.3	Classification Reports for Component-specific Models.	61
Figure 4.3.4	Class Distribution before and after Applying SMOTE.	61
Figure 4.3.5	Precision-Recal Curves (Before vs After).	61
Figure 4.3.6	Top 10 Most Important Features.	62
Figure 4.3.7	Real-time System Telemetry Collection using HWiNFO.	63
Figure 4.3.8	Easy to Read Proof of Collection using OHM.	63
Figure 4.5.1	User Upload CSV File.	65
Figure 4.5.2	User Download Script to run Model Testing.	65
Figure 4.5.3	JSON Output received from user.	66
Figure 4.5.4	HTML Report received from user.	66
Figure 4.6	Processing Time from Real-time Telemetry to Output.	67

LIST OF TABLES

Table Number	Title	Page
Table 1.5	Key Terms and Definitions Used in Predictive Maintenance for Server Failure	21
Table 3.1.1	Specifications of laptop	38
Table 3.2.1	Modular Block of Core Pipeline	45
Table 3.2.2	Summary of Datasets Used	46
Table 3.2.3	Mapping of Sensor Metrics to Hardware Component	48
Table 3.2.4	Equation used for Interaction Features of Model Testing	51
Table 4.1	Software Tools used for Predictive Maintenance	55
Table 4.5	User Types and Their Actions	65

LIST OF SYMBOLS

β	Constant coefficient (for CPU failure model)
γ	Constant coefficient (for Memory failure model)
T	Temperature (°C)
U_{cpu}	CPU utilization (%)
U_{ram}	RAM utilization (%)
t_{up}	System uptime (hours)
τ_{temp}	Temperature threshold (75°C)
τ_{cpu}	CPU utilization threshold (90%)
τ_{up}	Uptime threshold (5000 hours)
τ_{ram}	RAM usage threshold (85%)
V	Voltage (V)
V_{min}	Minimum safe voltage (11.5V)
V_{max}	Maximum safe voltage (12.5V)
$I(\cdot)$	Indicator function (returns 1 if condition is true, 0 otherwise)
P_{cpu}	Probability of CPU failure
P_{mem}	Probability of memory failure
P	Power (W)
τ	Torque (Nm)
ω	Angular velocity (rad/s)
N	Rotational speed (rpm)
R_{TV}	Temperature-voltage ratio (°C/V)
μ_t	Moving average at time t
σ_t	Standard deviation at time t
x_i	Observed value at time i
t	Current time index
$Risk(x)$	Risk classification level
$P(x)$	Model-predicted failure probability for observation x
θ	Model parameters

L	Cross-entropy loss function
$\Omega(\theta)$	Regularization term
f_{θ}	Model function parameterized by θ
y_i	True label of observation i
τ^*	Decision threshold
$\hat{y}$	Predicted failure label (1 or 0)
W_{τ}	Tool wear–torque interaction term
w	Tool wear (minutes)

LIST OF ABBREVIATIONS

<i>PM</i>	Predictive Maintenance
<i>OHM</i>	Open Hardware Monitor
<i>HWINFO</i>	Hardware Information Tool
<i>SMOTE</i>	Synthetic Minority Over-sampling Technique
<i>ML</i>	Machine Learning
<i>AI</i>	Artificial Intelligence
<i>ANN</i>	Artificial Neural Network
<i>LSTM</i>	Long Short-Term Memory
<i>AUC</i>	Area Under Curve
<i>CPU</i>	Central Processing Unit
<i>GPU</i>	Graphics Processing Unit
<i>RAM</i>	Random Access Memory
<i>NIC</i>	Network Interface Card
<i>IC</i>	Integrated Circuit
<i>S.M.A.R.T.</i>	Self-Monitoring, Analysis, and Reporting Technology
<i>CSV</i>	Comma Separated Values
<i>RUL</i>	Remaining Useful Life
<i>SVM</i>	Support Vector Machine
<i>API</i>	Application Programming Interface
<i>ROC</i>	Receiver Operating Characteristic

CHECKLIST

TICK (√)	DOCUMENT ITEMS
	Your report must include all the items below. Put a tick on the left column after you have checked your report with respect to the corresponding item.
√	Title Page
√	Copyright Statement
√	Acknowledgement
√	Abstract (with Area of Study and Keywords)
√	Table of Contents
√	List of Figures (if applicable)
√	List of Tables (if applicable)
√	List of Symbols (if applicable)
√	List of Abbreviations (if applicable)
√	Chapters / Content
√	References
√	All references are cited in the report, especially in the chapter of literature review
√	Appendices (if applicable)
√	Poster
√	I agree 5 marks will be deducted due to incorrect format, declare wrongly the ticked of these items, and/or any dispute happening for these items in this report.

CHAPTER 1

Introduction

As businesses increasingly depend on virtualized infrastructures for cloud computing, data handling, and day-to-day operations, maintaining server reliability has become more important than ever. These infrastructures use technologies like virtual machines and resource pooling to boost efficiency and scalability [1], [2]. However, as systems grow more complex—with different types of processors, workloads, and environments—keeping them running smoothly becomes harder, especially across diverse setups [7].

Unexpected server failures can cause serious issues, especially in industries like finance, telecommunications, and cloud services, where uptime directly impacts income and customer satisfaction [3], [4]. For instance, a single server outage could cost a company around \$5,600 for every minute it's down, showing how essential it is to have a maintenance plan that can stop problems before they happen [3].

One effective solution is predictive maintenance (PM), which uses artificial intelligence (AI) and machine learning (ML) to spot signs of potential failures early on by analyzing real-time and historical data [5], [6]. These systems pull information from sensors, system logs, and workload data to predict when something might go wrong and help prevent it with timely actions.

This project introduces an AI-based predictive maintenance system that works with various server types and configurations. To make the model reliable and adaptable, it uses several real-world datasets that cover different processor brands, failure types, hardware indicators, and usage conditions [8]. For example, the Laptop Motherboard Health Monitoring, Microsoft Azure Predictive Maintenance, and Google Cluster Workload datasets help the model learn from different processor types and usage patterns. Meanwhile, datasets like C-MAPSS, AI4I 2020, and Sensor-based Machine Failure provide important physical sensor data such as temperature and power use. The Hard Drive Test Data is also used to train the model on more specific hardware failure scenarios. By combining all these sources, the system is better equipped to detect possible problems early, no matter the setup.

CHAPTER 1

However, due to the sensitive nature and unavailability of much of the required historical server failure data, this project relies on real-time data (7-14 days) and verified simulated data for testing and model validation. This approach allows for preventive testing while accounting for the challenges in acquiring comprehensive historical data.

To make the model even more accurate, live sensor data is collected using tools like HWiNFO, Open Hardware Monitor (OHM), and the Zabbix Agent. These tools track values such as CPU temperature, voltage, fan speed, and system load, and the results are saved in CSV files to simulate real-time input for the model.

The machine learning pipeline is built using Python and includes several steps: importing libraries, loading and cleaning data, validating it, creating useful features, training and testing models, interpreting the results, and preparing the system for deployment [9]. The final output provides reports in formats like JSON and HTML, helping administrators decide whether a server is "at risk" or "stable." Overall, this system aims to reduce unexpected downtime, lower maintenance costs, and keep servers running smoothly in a wide variety of setups.

1.1 Problem Statement and Motivation

Problem Statement

As modern organizations expand their reliance on virtualized and cloud-based infrastructures, ensuring continuous server uptime has become increasingly complex. These environments often involve a wide range of server configurations, including different processor architectures (Intel, AMD, Apple, etc.), varying workloads, and diverse hardware setups. Traditional maintenance strategies, such as periodic checks or reactive repairs, are no longer sufficient in such dynamic environments. They often fail to detect early warning signs of system failure, leading to unexpected downtime that can severely impact business continuity and service availability.

While some predictive maintenance (PM) solutions do exist, many are limited in scope—either tailored to specific hardware setups, lacking support for real-time monitoring, or built on narrowly focused datasets that don't generalize well across different systems. This results in inaccurate predictions or an inability to detect failures before they occur, particularly in heterogeneous server environments. Moreover, many small to mid-sized organizations lack access to an integrated PM system that combines historical and real-time data, sensor inputs, and scalable AI capabilities. The absence of a flexible, adaptable, and accurate predictive maintenance system remains a significant gap in server infrastructure management today.

Motivation

The motivation behind this project stems from the growing demand for smarter and more adaptable predictive maintenance solutions that can effectively reduce server downtime, optimize resource usage, and improve system reliability across a wide range of computing environments. As enterprises continue to scale their infrastructure with cloud and edge computing technologies, the complexity of server systems is increasing—requiring more intelligent solutions that go beyond traditional static monitoring. Developing a robust predictive maintenance model that leverages both historical and real-time data offers a proactive approach to managing this complexity.

This project aims to bridge the gap between current maintenance practices and the actual operational needs of modern IT environments. By utilizing a comprehensive range of open-source datasets—including those from Microsoft Azure, Google Cluster, AI4I 2020, and C-MAPSS—the model will learn from a variety of failure types, hardware metrics, and processor configurations. This diversity enhances the model's generalizability and effectiveness in predicting failures across different platforms. In addition to historical data, the integration of real-time sensor monitoring tools such as HWiNFO, Open Hardware Monitor, and Zabbix Agent allows for continuous data collection, ensuring that the model can adapt to live server conditions and alert administrators before critical failures occur.

From a personal and academic standpoint, this project also serves as an opportunity to explore how AI and machine learning can be practically applied to system maintenance—a crucial but often overlooked aspect of IT operations. By developing a scalable, Python-based pipeline that supports data ingestion, preprocessing, model training, evaluation, and deployment, the project contributes a novel and practical solution to the field of intelligent infrastructure management. Ultimately, this work aspires to minimize unplanned outages, improve the efficiency of server management, and serve as a stepping stone for future research in predictive maintenance across broader IT ecosystems.

1.2 Research Objectives

The main goal of this project is to design and develop an AI-driven predictive maintenance system that can accurately predict server failures in various operating environments. The system will focus on building a machine learning (ML) model capable of analyzing both historical and real-time hardware sensor data to forecast potential server issues before they occur. This proactive approach aims to help reduce server downtime, prevent data loss, and optimize system performance in heterogeneous IT environments.

The project will focus on integrating and processing data from different types of sources, including public datasets and live sensor readings collected through tools such as HWiNFO, Open Hardware Monitor (OHM), and Zabbix Agent. These sources will provide important hardware metrics like temperature, voltage, and fan speed, which are essential for understanding the condition and behavior of servers over time. The system will also use various publicly available datasets that cover different processor types, failure patterns, and workload conditions. This diversity will help the model generalize well to different server configurations. This project will specifically aim to:

1. Build a reliable machine learning pipeline in Python, hosted within Jupyter Notebook, that includes data preprocessing, feature engineering, model training, and result visualization.
2. Achieve at least 80% accuracy in predicting server failure risks based on defined performance metrics like precision, recall, and F1-score.
3. Simulate real-time prediction scenarios using batch processing of sensor data, allowing users to visualize server health status through dashboards and risk classification outputs in HTML and JSON formats.
4. Demonstrate the feasibility of using AI-based techniques in predictive maintenance without requiring specialized or expensive hardware infrastructure.

However, this project will not cover areas such as analyzing log files, incorporating visual diagnostics (e.g., thermal imaging), or supporting multilingual outputs. It also won't deliver a full-scale enterprise-ready system but rather a working prototype that proves the concept. Scalability, deployment in cloud environments, and continuous real-time prediction pipelines are considered potential future extensions, not current deliverables.

CHAPTER 1

In short, this project aims to bridge the gap between raw server data and intelligent decision-making using AI, focusing on building a foundation that can later be enhanced and expanded to support broader and more complex IT infrastructure environments.

1.3 Project Scope and Direction

Project Scope

This project aims to create an AI-based predictive maintenance system that can predict server failures before they happen. The system will use both historical and real-time sensor data to make accurate predictions, helping system administrators take early action and avoid unexpected downtime. To keep the project realistic within the time limit of a final year project, a few scope limitations and clear deliverables are set.

The focus will be on servers that use common x86 processor types like Intel and AMD. Other processor types such as Apple ARM and Nvidia will be included only in simulated datasets for testing the model's flexibility. For collecting real-time data, the project will use tools like HWiNFO, Open Hardware Monitor (OHM), and Zabbix Agent in a Windows environment. These tools will collect important sensor data such as CPU temperature, voltage, and fan speed, and export them in CSV format for the system to analyze. The whole system will be developed using Python and built inside Jupyter Notebook because it's easy to use for data analysis, testing, and visualizing results. Below are some of the main features of the system:

1. Accurate Failure Prediction

The model should be able to predict server failures with at least 80% accuracy. This goal helps show that the model works well and can be trusted. The performance will be measured using standard metrics like precision, recall, and F1-score.

2. Real-Time Data Simulation

Although the system won't run fully in real-time, it will simulate real-time conditions by reading hardware sensor data at intervals. This shows that in the future, the system could be fully integrated with live servers.

3. User-Friendly Outputs

The system will provide results in different formats, such as HTML dashboards, JSON files, and easy-to-understand graphs. This will help users quickly know whether a server is "at risk" or "stable."

4. Use of Multiple Datasets

CHAPTER 1

The model will be trained using a variety of public datasets like Microsoft Azure Predictive Maintenance, C-MAPSS, AI4I 2020, and Google Cluster Workload. These datasets include different types of hardware, failures, and usage patterns to help make the model more reliable and versatile.

The project will not focus on building a large-scale enterprise-ready system or supporting huge server networks. Also, it will only use numerical data like sensor readings, and not system logs or images. Everything will be presented in English only. However, this project sets a solid base that can be expanded and improved in future versions.

Direction

This project is the first step toward building a more advanced and scalable predictive maintenance system that could be used in real-world businesses or data centers. While the current version only simulates real-time conditions and runs in a simple environment, future upgrades could allow the system to work continuously with real-time data from cloud platforms or large networks.

Later on, the system could be improved by adding deep learning techniques, better user interfaces, and more types of input such as server logs or network data. It could also be made available in other languages and deployed using cloud or container technologies like Docker, making it easier to scale and use in different environments.

Overall, the direction of this project is to become a smart, flexible system that helps reduce server downtime and improve performance across different types of hardware and organizations.

1.4 Contributions

The contribution of this project aims to provide an effective, intelligent, and accessible solution to help organizations predict server failures before they occur. By implementing a machine learning-based predictive maintenance system, this project supports IT teams in reducing downtime, minimizing operational disruptions, and improving system reliability. Below are the three main contributions this project offers:

1. Accurate and Early Failure Prediction

The system is designed to analyze both real-time and historical data collected from server sensors to detect anomalies and predict failures before they happen. The machine learning model is trained on diverse datasets involving different hardware configurations and failure scenarios, which helps the system achieve high prediction accuracy (targeted at 80% or above). This allows administrators to act early and avoid costly system crashes or service interruptions.

2. Easy Integration and Visualization

To support ease of use and adoption, the system will be deployed within a Jupyter Notebook environment and include clear visualizations such as graphs, dashboards, and HTML reports. These outputs provide intuitive insights into server health and risk levels. The structured JSON outputs can also be integrated into other IT management tools if required. Tools like HWiNFO, OHM, and Zabbix Agent will be used for data collection, making the system practical and cost-effective to implement on existing Windows-based infrastructures.

3. Generalizability Across Server Platforms

The project utilizes multiple public datasets that represent different processor types (Intel, AMD, etc.), workloads, and hardware conditions, which helps improve the model's ability to generalize across heterogeneous server environments. While real-time data collection will focus on x86-based systems, the project's architecture allows future extension to other platforms. This shows that the system has the potential to be scaled or adapted to different server types and industries.

1.5 Background Information

In today's digital era, server infrastructure forms the backbone of nearly all modern enterprises, cloud services, and data-intensive applications. Ensuring the reliability and uptime of servers is critical, as unexpected hardware failures can result in costly downtime, data loss, and service interruptions. Traditional maintenance strategies, such as reactive maintenance (fixing components only after failure) and scheduled preventive maintenance, are often inefficient or insufficient in complex, high-demand environments. This has led to the rise of Predictive Maintenance (PM) — an approach that leverages data-driven insights to anticipate and prevent failures before they occur. Predictive maintenance uses machine learning models to analyze historical and real-time sensor data, enabling early detection of component degradation or failure risks.

Predictive maintenance techniques originally gained traction in fields such as manufacturing, industrial machinery, and aerospace (e.g., the C-MAPSS turbofan engine dataset is a classic example). However, applying similar techniques to IT infrastructure and server hardware is a relatively new and evolving domain. Server environments present unique challenges — with diverse processor architectures, complex sensor telemetry (temperature, voltage, power usage, etc.), and multiple failure types (hardware, software, network). Recent advancements in telemetry monitoring tools (like HWINFO, OpenHardwareMonitor, and Zabbix) have made it feasible to collect detailed server health data. Yet, real-world server failure datasets remain scarce and sensitive, limiting accessibility and slowing research progress in this area.

This project bridges concepts from industrial predictive maintenance, machine learning, and server infrastructure monitoring to build a comprehensive predictive maintenance system tailored to servers. The system integrates data from multiple sources — publicly available datasets, real-time sensor data, and simulated failure scenarios — to develop machine learning models capable of predicting server hardware risks. Key technical concepts include feature engineering (extracting meaningful metrics like temperature-voltage ratios), failure pattern analysis, risk classification, and deployment of prediction pipelines for practical use. Readers unfamiliar with predictive maintenance should understand that PM systems aim to shift maintenance strategies from reactive to proactive — minimizing downtime and optimizing server reliability through timely, data-informed interventions. This report will guide the reader through both the

CHAPTER 1

theoretical foundation and practical implementation of such a system, even for those without deep experience in server management or machine learning.

Term	Definition
Predictive Maintenance (PM)	A maintenance strategy that uses data analysis tools and techniques to detect anomalies and predict equipment failures before they occur.
Telemetry Data	Automatic collection and transmission of measurements and operational data from hardware devices to a central system for monitoring and analysis.
Open Hardware Monitor (OHM)	A free and open-source tool that monitors temperature sensors, fan speeds, voltages, and load of hardware components in a computer system.
HWINFO	A comprehensive hardware monitoring and diagnostic tool that provides in-depth telemetry on CPU, GPU, memory, drives, and motherboard sensors.
Zabbix Agent	A lightweight software component that collects monitoring data (e.g., CPU usage, disk space, network activity) from servers and sends it to a Zabbix monitoring server.
Failure Types	Categories of possible system failures, such as hardware faults (CPU, disk), software crashes, and network disruptions.
Feature Engineering	The process of transforming raw data into meaningful input features that improve the performance of machine learning models.
SMOTE (Synthetic Minority Over-sampling Technique)	A method used to balance class distribution in imbalanced datasets by generating synthetic samples of the minority class.
Concept Drift	A phenomenon where the statistical properties of target variables change over time, potentially degrading model performance if not updated.
Hyperparameter Tuning	The process of optimizing parameters that govern the learning process of machine learning models to achieve the best performance.

Table 1.5 Key Terms and Definitions Used in Predictive Maintenance for Server Failure

1.6 Report Organization

The details of this research are presented in the following chapters. Chapter 2 reviews the background on predictive maintenance, machine learning techniques, and relevant data collection tools like HWiNFO, OHM, and Zabbix Agent. Chapter 3 outlines the methodology for building the predictive maintenance system, including data collection, preprocessing, and feature engineering. Chapter 4 discusses the implementation of the machine learning pipeline and the integration of real-time and historical data. Chapter 5 presents the results, including model evaluation and performance metrics. Finally, Chapter 6 concludes with a discussion on limitations, future improvements, and potential directions for the system.

CHAPTER 2

Literature Reviews

2.1 Prior Arts in Predictive Maintenance for Server Failure

2.1.1 Predictive Maintenance in Server Infrastructure

Predictive maintenance (PM) has become a pivotal approach for ensuring the reliability and efficiency of server infrastructure, particularly in large-scale data centers and cloud platforms. By leveraging real-time monitoring and intelligent data analysis, PM shifts maintenance practices from reactive or scheduled to proactive, significantly reducing downtime and costs.

Many leading organizations, including Google, AWS, and Microsoft Azure, have integrated AI-driven predictive models into their infrastructures to detect anomalies early and enhance fault resilience [13]. These implementations utilize machine learning (ML) and deep learning models—such as LSTM, Transformers, and Artificial Neural Networks (ANN)—to forecast failures in servers, storage systems, and network components using telemetry data like CPU usage, memory load, and disk I/O [13], [14], [15]. ANN, in particular, has demonstrated high accuracy in predicting hardware failures and estimating remaining useful life (RUL) [14].

Studies also highlight the application of evolving fuzzy-rule-based systems and hybrid ML techniques, such as One-Class SVMs and Elastic Stack Suites, for analyzing log streams and clustering anomalies [10], [11]. These methods enhance early fault detection and support preventive maintenance strategies. Moreover, the rise of edge computing allows part of the predictive workload to be processed near the data source, reducing latency and enabling faster responses to impending issues [12].

In summary, modern predictive maintenance frameworks combine diverse AI and ML techniques with edge computing and telemetry systems to deliver intelligent, self-healing infrastructures. These innovations are transforming server maintenance into a strategic function crucial for the resilience and scalability of mission-critical environments.

2.1.2 Hardware Health Monitoring

Monitoring hardware health is central to predictive maintenance strategies in server environments, where failures in components like CPUs, memory, disks, or GPUs can result in substantial operational costs. Academic studies underscore the need for real-time surveillance of system parameters to detect early signs of degradation. For instance, Decker et al. [10], [11] and Kumar [12] emphasize AI-assisted monitoring using server logs and sensor data, which improves fault detection by identifying anomalies in performance metrics such as CPU usage, memory load, and disk activity.

Telemetry systems are instrumental in gathering the data necessary for training predictive models. Research by Polu [13], BOUROGA and SAHRAOUI [14], and Gao et al. [15] collectively highlights how telemetry from critical resources informs machine learning algorithms for failure forecasting. These systems contribute to enhanced accuracy and responsiveness in detecting potential breakdowns.

On the practical side, Chapman [16] and Aldea et al. [17] provide valuable perspectives on commonly used monitoring tools like HWiNFO, Open Hardware Monitor (OHM), and Zabbix. These utilities collect detailed hardware metrics and are often integrated with higher-level platforms to support proactive decision-making.

Edge computing also plays a growing role in accelerating hardware health analytics. Mourtzis et al. [18] describe an edge-computing platform leveraging 5G and machine learning to minimize latency and improve fault prediction, further supporting real-time decision-making and reducing downtime.

In essence, effective hardware health monitoring combines academic methodologies, real-time telemetry, and industry tools within intelligent architectures that underpin predictive maintenance and ensure sustained server performance.

2.1.3 Techniques for Failure Prediction

Failure prediction in server systems has undergone a transformation from traditional statistical methods to sophisticated AI-driven models. Early techniques, rooted in reliability engineering and statistical modeling—such as multivariate regression,

ARIMA, and Hidden Markov Models (HMM)—focused on predefined thresholds and historical failure trends [13], [14]. While foundational, these approaches often struggle in dynamic and heterogeneous data center environments.

Machine learning (ML) techniques introduced a more adaptive framework. Models like Decision Trees, Support Vector Machines (SVM), and Random Forests analyze telemetry and log data to identify early warning signs of system degradation. Both supervised and unsupervised approaches enable these models to handle varied operational scenarios, improving fault detection accuracy and response [11], [13], [14]. Clustering methods further support anomaly detection when labeled data is sparse.

The progression to deep learning (DL) has further enhanced predictive capabilities. Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, and Transformer-based models can extract complex temporal and spatial relationships from high-dimensional telemetry datasets [12], [13], [15]. These architectures are particularly effective in detecting subtle, evolving patterns that precede failures, making them well-suited for real-time and large-scale environments.

Hybrid and fuzzy-logic-based systems represent another evolution, offering interpretability and adaptability. By combining evolving rule-based mechanisms with learning algorithms, these approaches manage uncertainty and update models continuously based on live data. Their use in real-time anomaly detection demonstrates practical effectiveness in mission-critical settings [10], [12], [18].

Collectively, the literature reveals a clear trend: from rule-based systems toward intelligent, self-adaptive models capable of anticipating failures in complex infrastructures. This evolution supports more resilient and autonomous server maintenance strategies, ultimately driving efforts toward achieving zero-downtime operations.

2.2 Datasets and Data Collection for Predictive Maintenance

2.2.1 Processor & Motherboard Datasets

The Laptop Motherboard Health Monitoring Dataset by Zia [20] offers a synthetic dataset focused on detecting hardware issues in laptop motherboards. It includes key features like CPU usage, voltage, temperature, RAM, and fan speed across 2000 records labeled with specific failure types. While useful for educational experiments and machine learning prototyping, the dataset lacks real-world noise and is not representative of enterprise server environments.

Another widely used resource is the Microsoft Azure Predictive Maintenance Dataset [21], which consists of telemetry, error logs, and maintenance history for 100 industrial machines. It has been employed to model fault prediction using multivariate time-series data. Azab et al. [19] utilized this dataset in their discrete-event simulation model to optimize scheduling and maintenance in industrial systems. However, the dataset is tailored to manufacturing rather than IT hardware, and its application to server-level predictive maintenance is indirect at best.

2.2.2 Sensor & Hardware Metrics Datasets

The C-MAPSS Dataset from NASA [22] simulates engine degradation over time using sensor data across different operational conditions. It is ideal for remaining useful life (RUL) prediction and widely adopted in benchmark studies. However, it does not represent server hardware metrics, limiting its relevance to IT predictive maintenance.

Similarly, the AI4I 2020 Predictive Maintenance Dataset [23] from UCI is a synthetic dataset for tool failure prediction using process temperature, torque, and wear data. Though well-labeled and useful for testing classification models, it is based on a manufacturing machine and offers no direct applicability to server components.

The Machine Failure Prediction Dataset by Naeem [24] introduces multiple sensor types, including VOC, ultrasonic, and environmental measurements. While versatile for general machine failure modeling, it lacks domain-specific details for server infrastructure.

2.2.3 Failure Type Datasets

The Backblaze Hard Drive Dataset [25] is a rare real-world dataset that provides daily S.M.A.R.T. metrics and failure records for thousands of hard drives in data center environments. It enables long-term modeling of drive reliability but is limited to storage components, with no data on CPUs, motherboards, or network devices. Currently, there is a significant lack of publicly available datasets on network hardware failures such as switches and NICs. Sensitive operational data and proprietary concerns likely hinder dataset release in this domain, marking it as a key research opportunity.

2.2.4 Usage Pattern Datasets

Usage pattern datasets capture how systems are operated over time, which can provide vital context for predicting failures. One notable dataset is the Google Cluster Trace [26], which contains over 29 days of usage data from Google's data centers, including job arrival times, resource usage, and machine events. It has been used in workload modeling and anomaly detection research. While rich in operational detail, it lacks specific hardware failure labels, making it more suitable as a supplementary dataset for correlating workload patterns with maintenance events.

2.3 Fact Finding and Data Analysis Techniques

2.3.1 Scientific Methods in PM Data Analysis

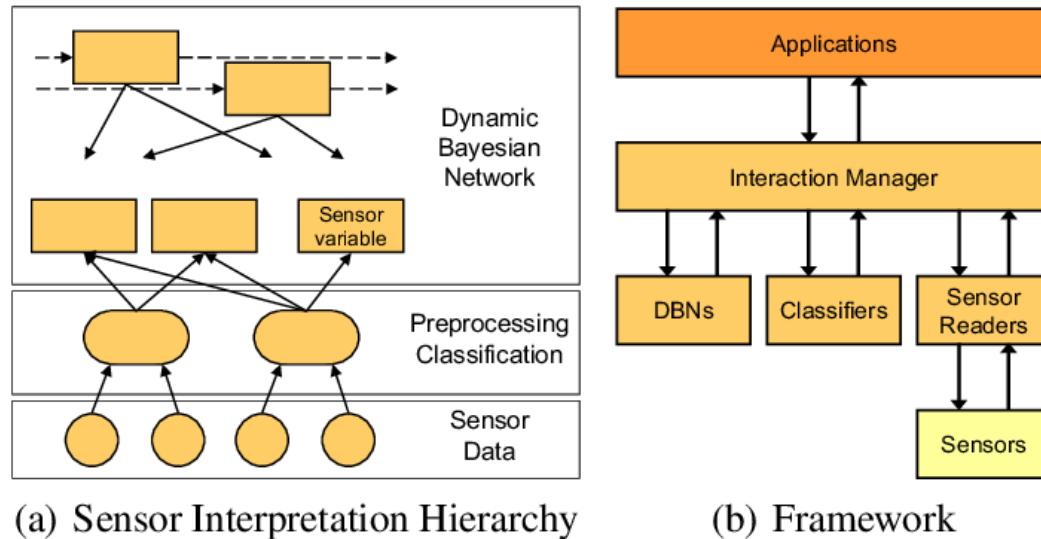


Figure 2.3.1: Sensor Data Flow

In predictive maintenance (PM), applying scientific and structured methods is critical to ensure accurate analysis of system health. Traditional practices often involve following well-defined maintenance procedures and system documentation, but modern techniques integrate advanced models such as Dynamic Bayesian Networks to interpret sensor data in real-time, as shown in Figure 2.3.1. Brandherm and Schmitz [27] proposed a modular framework that utilizes these networks for interpreting sensor data on mobile devices, highlighting how structured probabilistic models can support data-driven diagnostics. While the study focuses more on mobile systems, the methodology demonstrates how structured data interpretation methods can be effectively applied in PM scenarios. Additionally, Carvalho et al. [28] conducted a comprehensive review of machine learning (ML) techniques used in predictive maintenance, emphasizing the systematic process of data acquisition, preprocessing, and model development. Their study reinforces the importance of a scientific approach in data analysis to enable proactive fault detection and minimize equipment downtime. These techniques represent a modern evolution of traditional manual diagnostics, aligning well with the digital transformation goals in server infrastructure maintenance.

2.3.2 Data Element Analysis

Analyzing specific hardware components such as CPU, GPU, and RAM plays a central role in predictive maintenance, particularly in data center environments where performance monitoring is crucial. Several studies have highlighted the importance of evaluating these metrics to detect early signs of failure. For example, Decker et al. [10], [11] developed an evolving fuzzy-rule-based approach for real-time anomaly detection using log data, showing that tracking CPU and RAM performance can lead to effective predictive models. System architecture and data flow diagrams are often employed in these studies to visualize interactions between components, aiding in the interpretation of complex systems. Kumar [12] and Polu [13] further demonstrated that sensor data, when mapped through structured architectural diagrams, can identify bottlenecks and fault-prone components. Moreover, Bourouga and Sahraoui [14] applied multivariate regression techniques to telemetry data, emphasizing the predictive power of monitoring physical parameters that mirror server-side metrics like voltage and thermal output. Studies by Gao et al. [15] and Aldea et al. [17] also confirm that real-time monitoring of hardware resources combined with structured input-output analysis is essential for ensuring reliable performance. The consistent use of architecture diagrams and metric-based logging underlines a common framework for interpreting hardware health and forecasting failures in server ecosystems.

2.3.3 Report Use Analysis

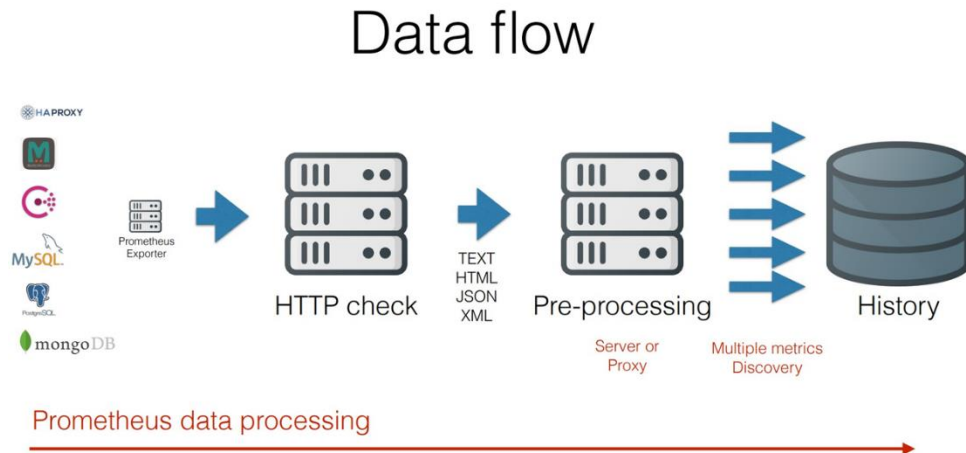


Figure 2.3.2: Zabbix Data Flow

Diagnostic reports generated by tools such as HWiNFO, Open Hardware Monitor (OHM), and Zabbix play a vital role in predictive maintenance (PM) by enabling continuous monitoring and analysis of system performance metrics. These tools provide detailed insights into hardware resource usage, environmental conditions, and potential system anomalies, supporting proactive failure prevention strategies. Zabbix, in particular, has seen enhancements in its recent versions, including machine learning integration and predictive functions that allow automated incident detection and forecasting based on monitored data trends [29]. Reports generated through Zabbix enable system administrators to identify early warning signs of hardware issues, optimize resource allocation, and plan maintenance activities before critical failures occur [30]. Figure 2.3.2 illustrates the typical data flow for such diagnostic tools, from raw HTTP metrics to processed insights.

Moreover, the use of automated reporting and root cause analysis within Zabbix helps streamline troubleshooting processes, improving the overall reliability and efficiency of IT infrastructure. Zabbix also allows for the integration of diverse data sources, making it effective for comprehensive system analysis. Such data can be used not only for operational decision-making but also to inform stakeholders about potential risks and the general health of the infrastructure [29], [30].

Complementing these capabilities, tools like HWiNFO are crucial for environmental monitoring. They continuously log electrical parameters and thermal conditions, which

are critical factors influencing server hardware performance and longevity. Reports from these tools can reveal trends, such as rising component temperatures or voltage irregularities, that may indicate a developing issue [31]. Additionally, diagnostic stress testing using tools like AIDA64 and Prime95 enables engineers to assess component behavior under load, supporting the identification of weak points that are susceptible to failure. This kind of testing and analysis helps determine the reliability of CPUs, GPUs, and memory modules under extreme operational scenarios [31].

Ultimately, combining stress testing, environmental monitoring, and automated diagnostic reporting provides a multi-faceted approach to predictive maintenance. These tools help technicians detect abnormal patterns early, understand root causes of failures, and schedule preventive actions effectively. Such integration is essential for maintaining server uptime, avoiding costly downtimes, and ensuring long-term system resilience.

2.4 Existing Predictive Maintenance Models and Techniques

2.4.1 Machine Learning Models in PM

Machine learning (ML) techniques have played a significant role in advancing predictive maintenance (PM) strategies in server and data center environments. Commonly applied models such as Support Vector Machines (SVM), Random Forest (RF), and Gradient Boosting algorithms have shown effectiveness in identifying early signs of hardware or system failures by analyzing telemetry and log data. For example, studies by Decker et al. [11], Polu [13], and Gao et al. [15] demonstrated how supervised learning approaches like SVM and RF can accurately predict machine failures in data centers. These models excel at handling structured datasets and offer interpretable outcomes, making them suitable for applications where explainability is essential. Additionally, Mourtzis et al. [18] highlighted the integration of ML models with edge computing platforms to calculate Remaining Useful Life (RUL) of components, further enhancing real-time decision-making capabilities. However, these models can face limitations when dealing with highly dynamic environments or unstructured data, and their performance often depends heavily on the quality and completeness of the input features.

2.4.2 Deep Learning Models

Deep learning (DL) models have emerged as powerful alternatives to traditional ML methods, especially for analyzing complex, high-dimensional data such as time-series telemetry and system logs. Models like Long Short-Term Memory (LSTM) networks and Transformer-based architectures are particularly adept at capturing temporal dependencies and subtle failure patterns over time. Studies by Kumar [12], Polu [13], and Gao et al. [15] demonstrated how LSTM networks can effectively learn from sequential data to enhance fault prediction accuracy. Transformer-based models have also been explored to improve performance in predictive maintenance tasks [13], [15]. Despite their superior ability to model complex data relationships, a notable challenge with deep learning approaches is the issue of explainability. Unlike traditional ML models, deep neural networks often operate as "black boxes," making it difficult to

interpret the rationale behind their predictions—a drawback that can hinder their adoption in critical infrastructure contexts where transparency is vital.

2.4.3 Hybrid Models

Hybrid models, which combine multiple data sources and analytical techniques, have gained traction in predictive maintenance due to their ability to leverage the strengths of different methods. These models typically integrate sensor data, environmental data, and workload or log data to refine predictions and improve system diagnostics. Studies by Decker et al. [11], Polu [13], and Mourtzis et al. [18] illustrated how combining diverse data streams can enhance the accuracy and robustness of predictive maintenance systems. Furthermore, Kumar [12] emphasized the role of edge computing in hybrid frameworks, which enables real-time analytics by processing data closer to its source. A practical example of hybrid model application can be found in the work of Azab et al. [19], where a pharmaceutical company case study validated the integration of ML-assisted simulation with sensor and environmental data for dynamic scheduling and predictive maintenance. Hybrid approaches are increasingly being recognized for their capacity to balance predictive accuracy, computational efficiency, and contextual adaptability in real-world industrial and server environments.

2.5 Critical Review of Previous Works

2.5.1 Strengths of Existing Approaches

Existing predictive maintenance (PM) approaches have demonstrated several notable strengths, particularly in terms of real-world applicability and model robustness. Several studies have successfully implemented PM frameworks in actual data center and industrial environments, illustrating the feasibility of their deployment beyond theoretical research. For instance, Decker et al. [10] and Decker de Sousa et al. [11] showcased machine learning and fuzzy-rule-based systems capable of performing real-time anomaly detection in data centers, demonstrating their potential for practical application. Additionally, many models presented in the literature are built on robust datasets, which contribute to their high accuracy and reliability. Techniques such as deep learning models (e.g., LSTM and Transformers) and advanced regression methods have been effective in improving predictive performance [13], [15]. Furthermore, approaches combining edge computing with AI, as explored by Kumar [12] and Mourtzis et al. [18], have shown the promise of reducing latency and enhancing real-time data processing capabilities, making these solutions highly relevant for modern data-intensive environments. The integration of machine learning into scheduling strategies, such as in dynamic flow-shop scheduling [19], also highlights how PM can be embedded into broader operational workflows, further enhancing system efficiency and equipment uptime.

2.5.2 Weaknesses and Gaps

Despite the strengths, there are several notable weaknesses and gaps in the current PM literature. One of the most critical limitations is the lack of server-specific predictive maintenance models. Much of the existing research tends to focus on industrial equipment, turbofan engines, or general data center infrastructure, with limited emphasis on server hardware components such as processors and motherboards [10], [11]. Moreover, many datasets used in these studies lack diversity and are often constrained to specific environments or case studies. For example, some works are limited to pharmaceutical production lines or single data centers, restricting the generalizability of the models [19].

Another prominent gap lies in scalability and adaptability. Many approaches, including those based on fuzzy-rule systems [10] and deep learning models [13], face challenges when handling large-scale, dynamic cloud or data center environments. Ensuring real-time processing without performance bottlenecks remains difficult across these studies [12], [15]. Additionally, the heavy reliance on high-quality and comprehensive datasets is a recurring issue. The effectiveness of many PM models diminishes when data is noisy, incomplete, or varies significantly between different systems [14], [15].

Explainability is also a persistent concern. While advanced AI models can achieve high accuracy, their decision-making processes are often opaque, making it difficult for administrators to trust and interpret predictions [13]. Finally, the resource-intensive nature of most approaches—both in terms of computational requirements and the need for expert knowledge to implement and maintain systems—poses a significant barrier to adoption, particularly in smaller organizations with limited budgets and expertise [12], [18].

2.5.3 Comparison with Proposed Solution

While existing predictive maintenance (PM) frameworks have demonstrated substantial advancements, particularly in large-scale server infrastructure, the proposed solution in this project offers several key improvements that aim to address the identified gaps in the literature. Previous studies have effectively integrated AI-driven models, such as LSTM, Transformers, and Artificial Neural Networks (ANN), to forecast failures by analyzing telemetry data like CPU usage, memory load, and disk activity [13], [14], [15]. However, many of these works rely on relatively narrow datasets that are often limited to single environments or specific case studies, which reduces model generalizability [19]. Furthermore, much of the focus has been on data center-wide infrastructures or industrial machinery, with less emphasis on individual server components such as processors, motherboards, and sensor-driven metrics [10], [11].

In contrast, the proposed workflow strategically integrates diverse datasets that encompass various processor types, hardware metrics, failure types, and usage patterns. The inclusion of datasets spanning from the Laptop Motherboard Health Monitoring Dataset and Microsoft Azure Predictive Maintenance Dataset, to C-MAPSS Turbofan

Engine Simulation and Google 2019 Cluster sample, ensures that the model captures a wide variety of configurations, operational scenarios, and failure behaviors. This broad dataset diversity enhances the robustness and transferability of the predictive models compared to those relying on a single source or homogeneous environment.

Another novel aspect of the proposed solution lies in its feature engineering process. While existing studies have used conventional telemetry data like CPU load and memory usage [13], [14], [15], this project extends feature engineering by incorporating derived metrics, rolling statistics, and hybrid features (such as temperature-voltage ratios and CPU-RAM interactions). These complex features aim to uncover deeper temporal and cross-component relationships, which can improve the early detection of hardware degradation. Previous works have acknowledged the benefit of evolving fuzzy-rule-based systems and hybrid ML models for anomaly detection [10], [11], but they rarely emphasize advanced, interpretable feature synthesis at this granularity.

Moreover, although current industry-standard monitoring tools like HWiNFO, Open Hardware Monitor, and Zabbix have been effectively integrated in practical settings for hardware health monitoring [16], [17], the proposed solution goes a step further by combining these real-time monitoring utilities with an end-to-end predictive model pipeline. This integration allows the system not only to monitor but also to analyze historical and near-real-time data, simulate different failure types, and provide actionable risk assessments. Existing literature also highlights the increasing role of edge computing for reducing latency in hardware health analytics [12], [18]; similarly, the proposed framework leverages real-time data streams and lightweight models to enable faster failure predictions directly from live data.

Finally, the comprehensive deployment readiness of the proposed system — with modular pipelines, risk classification outputs, interpretability modules, and example inference scripts — enhances its practicality and usability in real-world settings. This level of systematization and documentation is not always evident in prior research, which often stops short at model validation without extensive deployment preparation.

In summary, while prior works have laid a solid foundation for predictive maintenance in server infrastructures using ML and DL techniques, this project contributes by (1) integrating more diverse and representative datasets, (2) advancing feature engineering with derived and temporal features, and (3) offering a holistic system that combines

CHAPTER 2

real-time monitoring (HwiNFO, Zabbix) with an explainable, production-ready predictive model. These enhancements directly address limitations identified in the literature and support more resilient and adaptive server maintenance strategies.

CHAPTER 3

Proposed Method/Approach

This chapter outlines the method used to develop a predictive maintenance model for server failure prediction. It covers the system requirements, design, key equations, architecture, challenges, and project timeline, providing a clear framework for building and deploying the system.

3.1 System Requirement

3.1.1 Hardware

A laptop is utilized to develop and evaluate a predictive maintenance model within a virtualized environment, leveraging both real-time telemetry and verified simulated datasets to forecast server failures. The system is built through a comprehensive machine learning pipeline involving data preprocessing, feature engineering, model training, evaluation, and deployment preparation. Additionally, the laptop serves as the central platform for integrating hardware monitoring tools such as HWINFO, OHM, and ZabbixAgent, which supply the real-time data used for validating and refining the model's performance in practical scenarios.



Figure 3.1.1: Specifications of Laptop

Description	Specifications
Model	Lenovo Ideapad Gaming 3
Processor	AMD Ryzen 5 5600H with Radeon Graphics
Operating System	Windows 11 Home Single Language

Graphic	NVIDIA GeForce RTX™ 3050 Laptop GPU
Memory	16GB

Table 3.1.1: Specifications of laptop

3.1.2 Software



Figure 3.1.2: Zabbix

ZabbixAgent, a component of the Zabbix monitoring system, is employed in this project to simulate a scalable server monitoring setup. It collects system-level hardware metrics such as CPU utilization, memory usage, disk health, and network activity from the laptop and can be configured to push this data to a Zabbix server for centralized analysis. In the context of this project, ZabbixAgent is used to mimic the continuous monitoring framework typically deployed in enterprise server environments. Its integration allows for automated logging of real-time data, supporting the predictive maintenance pipeline by enabling consistent data collection for failure prediction and trend analysis.



Figure 3.1.3: HWiNFO

HWiNFO is a professional hardware information and diagnostic tool used in this project to collect real-time telemetry data from the laptop's internal sensors. It provides detailed, low-level data on system components such as CPU temperature, voltage, power usage, and fan speed. These metrics are critical for developing a predictive maintenance model, as they simulate the types of hardware indicators typically monitored in actual server environments. HWiNFO runs continuously in the

background and exports telemetry logs, which are then processed and integrated into the training dataset to model hardware degradation and failure risks over time.



Figure 3.1.4: OHM

Open Hardware Monitor (OHM) is an open-source tool utilized in this project to supplement hardware monitoring and cross-validate sensor data gathered by HWiNFO. It reads temperature sensors, voltage values, clock speeds, and load levels of the system's components. OHM is particularly useful for logging and visualizing real-time performance metrics in a lightweight manner. The data collected using OHM contributes to a robust telemetry dataset used during model training, feature engineering, and evaluation phases, ensuring the predictive model captures a diverse and accurate representation of hardware behavior under various conditions.

3.1.3 Data Processing Environment



Figure 3.1.5: Jupyter Notebook

Jupyter Notebook was used as the main platform for data processing and model development. Key libraries included Pandas and NumPy for data handling, Scikit-learn for machine learning, Matplotlib and Seaborn for visualization, SMOTE from Imbalanced-learn for handling class imbalance, and Datetime, JSON, and Joblib/Pickle for time-based features, input/output formatting, and model saving.

3.2 System Design

3.2.1 Architecture Development

This section outlines the system architecture developed for predictive maintenance of server failures. The design integrates real-time telemetry, public datasets, and machine learning to predict hardware risks. The following modular blocks break down the core pipeline: **Data Collection**, **Preprocessing**, **Feature Engineering**, **Model Training**, and **Inference**.

3.2.1.1 System Workflow

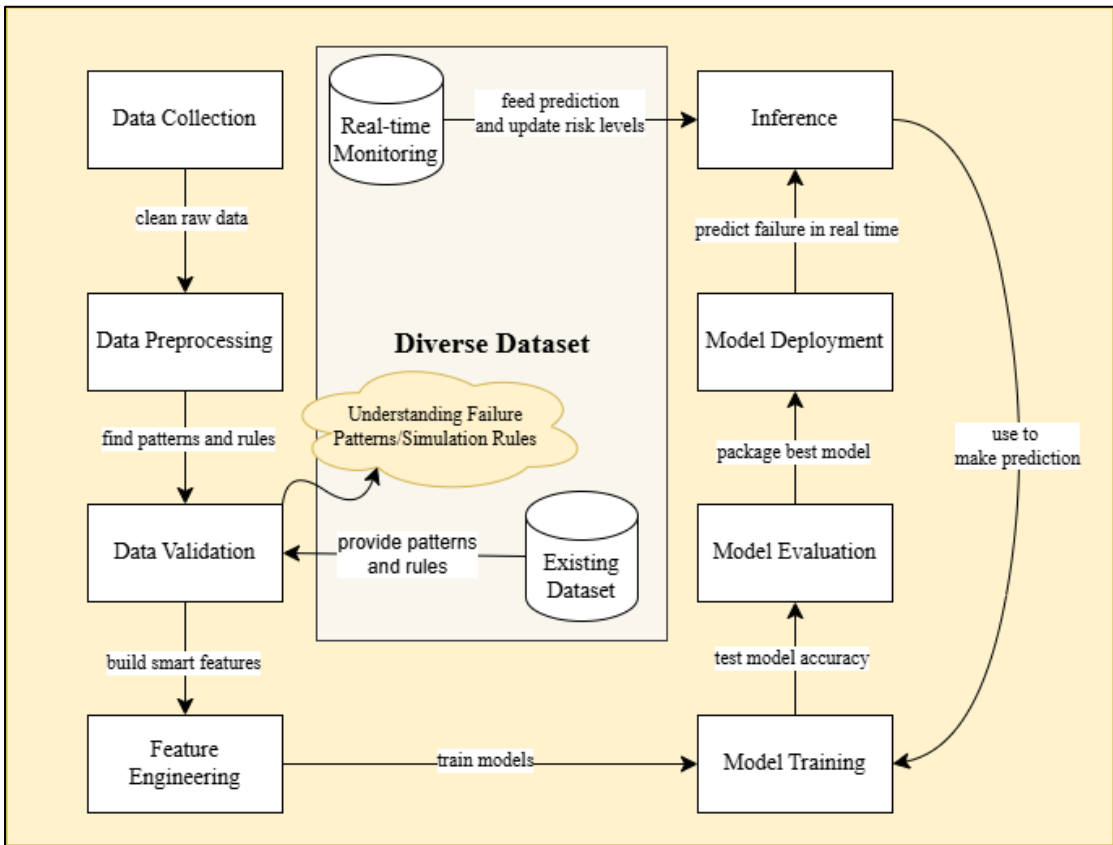


Figure 3.2.1: System Design Diagram

Block	Description
Data Collection	Curate datasets from various sources covering server health, sensor metrics, and failure events. Data sources include both publicly available datasets and custom-simulated logs representative of real-world server environments. Input formats include CSV, JSON, and native log outputs from monitoring tools.

Preprocessing	Clean and normalize data to ensure consistency. Tasks include handling missing values, timestamp alignment, normalization/scaling, removal of outliers, and standardization of feature names (e.g., mapping “CPU Temp” vs. “Processor Temp” to a canonical feature). Ensures all datasets conform to a unified schema to maximize model interoperability.
Feature Engineering	Derive meaningful predictors from raw sensor data. Example tasks include calculating rolling statistics (mean, variance), failure event lags, and domain-specific features (e.g., CPU temperature deltas, SMART disk parameters aggregation). Principal Component Analysis (PCA) or correlation-based feature selection may also be applied to reduce dimensionality where appropriate.
Model Training	Train multiple machine learning models per dataset (Random Forest, XGBoost, LSTM, etc.). Perform hyperparameter tuning and cross-validation. Save the best-performing model pipeline (model + preprocessing steps) for inference using tools such as joblib or pickle. Each dataset yields a modular, reusable model blueprint ready for deployment.
Inference & Output	When user-provided server data is input, the system selects compatible model pipelines, runs inference, and outputs results. Predictions are aggregated (e.g., via ensemble voting or weighted scoring). Output options include console display, log file, JSON/CSV report, and (optionally) dashboard visualizations.

Table 3.2.1: Modular Block of Core Pipeline

3.2.1.2 Dataset Summary Table

Dataset Name	Source	Size	Type	Relevance
Microsoft Azure Predictive Maintenance	Microsoft Open Datasets	~2 GB	Simulated	Generic asset monitoring
AI4I 2020 Predictive Maintenance	UCI Machine Learning Repository	~1 MB	Simulated	General predictive maintenance
Google Cluster Data	Google Research	~40 GB	Real	Server-specific workload & failures

Laptop Motherboard Health Monitoring	Kaggle	~200 KB	Synthetic	Hardware health monitoring for laptops (motherboard)
Hard Drive Test Data	Blackblaze	~15 GB	Real	Hard drive S.M.A.R.T. stats and failure prediction
NASA Turbofan Jet Engine Data Set	NASA	~3 GB	Simulated	Turbofan engine RUL prediction (failure degradation)
Machine Failure Prediction Using Sensor Data	Kaggle	~1 GB	Real	General machine failure prediction based on sensor data
Custom HWINFO Logs	Local Real-Time Monitoring	Varies	Real	Server-specific telemetry

Table 3.2.2: Summary of Datasets Used

The simulated datasets, such as the Microsoft Azure Predictive Maintenance and AI4I 2020 Predictive Maintenance, are valuable for testing predictive maintenance models across different industries and asset types. These datasets provide controlled environments where failure events are intentionally simulated, allowing for a well-defined testing ground. While these datasets are not tied to specific hardware, they offer a generalized view of how predictive maintenance models can be applied to various asset classes, making them useful for training models before applying them to real-world systems.

On the other hand, the real-world datasets, such as Google Cluster Data, Custom HWINFO Logs, and Zabbix Server Data, are directly relevant to your focus on server failure prediction. These datasets provide real telemetry data collected from actual servers, offering rich insights into the system's behavior in live operational environments. This data is essential for training and testing models that predict failures based on real-time metrics, such as CPU temperature, memory usage, and network traffic. The combination of these real-world datasets enhances the model's ability to generalize and identify failure indicators in servers and other critical hardware components.

Additionally, synthetic datasets like the Laptop Motherboard Health Monitoring Dataset and Machine Failure Prediction Using Sensor Data offer valuable insights for testing predictive maintenance algorithms in different contexts. These datasets focus on specific hardware components, such as laptop motherboards and various types of machinery, providing detailed sensor data (e.g., CPU usage, temperature, fan speed) that can be used to predict hardware failures. While not directly related to servers, they provide a different perspective on how sensor data can be leveraged to monitor and predict failures, which can be adapted to server maintenance.

The Hard Drive Test Data from Backblaze is another highly relevant dataset, particularly for predicting disk failures, which is a crucial aspect of server maintenance. This dataset includes detailed S.M.A.R.T. statistics and failure data from hard drives, which can be directly applied to the task of predicting disk degradation and failure in servers, thus enhancing the model's ability to address specific hardware components within server environments.

Lastly, the NASA Turbofan Jet Engine Dataset offers a unique opportunity to work with advanced degradation modeling data. While it focuses on jet engines, the principles of predicting remaining useful life (RUL) and handling fault evolution under various operational conditions can be directly applied to servers. This dataset helps improve predictive maintenance models by focusing on the progressive nature of component degradation and providing insights into handling sensor noise, which is often a challenge in real-world monitoring.

Together, these datasets provide a well-rounded foundation for developing a predictive maintenance system that can handle various hardware components, environments, and failure modes. Each dataset contributes a unique perspective that can help train and fine-tune models for real-time, actionable failure predictions across different systems.

3.2.1.3 Sensor Metric to Hardware Component Mapping Diagram

Sensor Metric	Hardware Component
CPU Temperature	CPU
GPU Temperature	GPU (if present)
Fan Speed	Cooling System (fans)

CHAPTER 3

Voltage (12V, 5V, etc.)	Power Supply Unit (PSU)
Disk SMART Attributes	Storage (HDD/SSD)
RAM Usage & Errors	Memory (RAM)
Network Packet Loss	Network Interface Card (NIC)
Motherboard Temp/Voltage	Motherboard

Table 3.2.3: Mapping of Sensor Metrics to Hardware Component

3.2.1.4 Data Flowchart

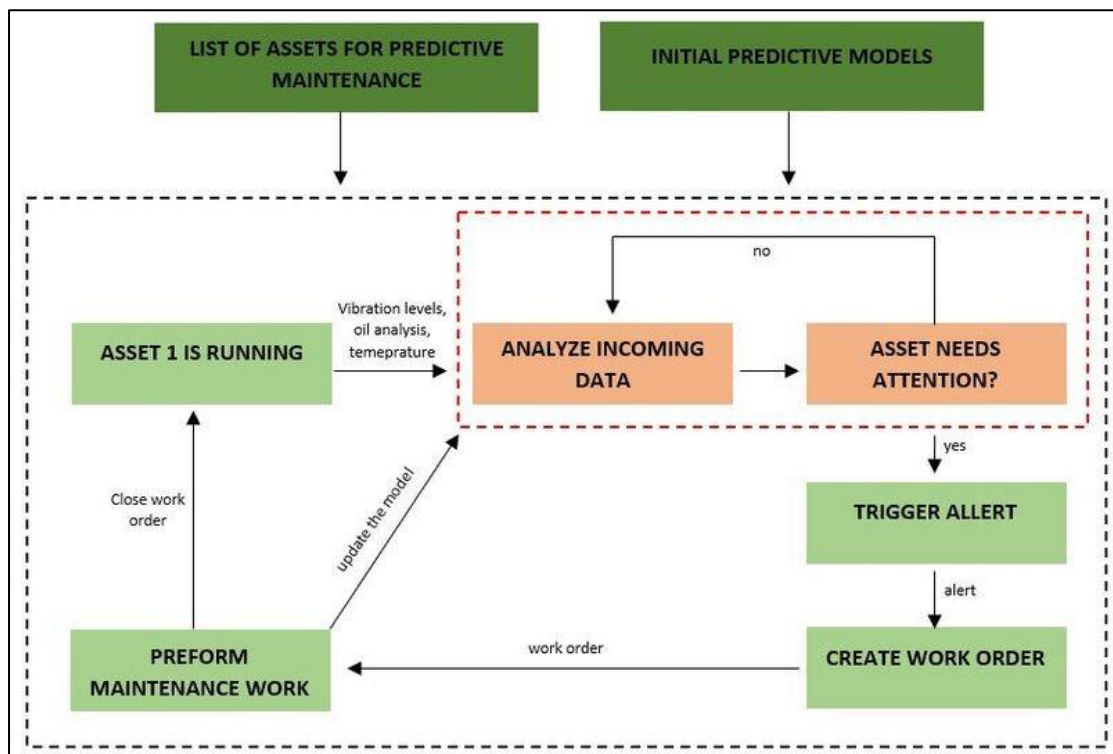


Figure 3.2.2: Predictive Maintenance Workflow

The predictive maintenance workflow depicted in the diagram closely aligns with and complements the comprehensive system design. The initial monitoring phase, which tracks key indicators like vibration and temperature, corresponds directly to the real-time data collection process in our system, where tools like HWINFO and ZabbixAgent continuously gather server health metrics. As incoming data is analyzed, our system performs preprocessing, validation, and feature engineering to transform raw metrics into meaningful insights, mirroring the workflow's data analysis stage. The critical decision point in the workflow - "ASSET NEEDS ATTENTION?" - is effectively implemented through our model's inference capabilities, where real-time predictions classify risk levels and trigger appropriate alerts. Finally, the workflow's maintenance actions, including work orders and system updates, are automated in our system through integration with monitoring platforms, enabling proactive responses to predicted failures. This seamless integration between the conceptual workflow and our technical implementation ensures a robust, end-to-end predictive maintenance solution that can effectively anticipate and prevent server failures.

3.2.2 System Equation

Feature/Function	Equation	Description
CPU Failure Probability	Eq. (1)	Estimates CPU failure likelihood based on high temperature (>75°C), high utilization (>90%), and long uptime (>5000 h).
Memory Failure Probability	Eq. (2)	Calculates memory failure risk based on high RAM usage (>85%) and abnormal voltage (outside 11.5V–12.5V).
Mechanical Power	Eq. (3)	Computes mechanical power from torque and rotational speed to capture load conditions relevant to stress.
Temperature-Voltage Ratio	Eq. (4)	Indicates thermal stress relative to voltage stability — useful as a stress feature.
Rolling Mean (n = 5)	Eq. (5)	Smooths recent sensor values over 5 -time steps to capture short-term trends and improve robustness.
Rolling Standard Deviation (n = 5)	Eq. (6)	Measures short-term variability in sensor values (window = 5). High variation may indicate instability.
Risk Classification Function	Eq. (7)	Converts predicted failure probability into qualitative risk levels for easier interpretation.
Model Training Objective (Random Forest)	Eq. (8)	Minimizes cross-entropy loss with regularization to balance fit and complexity in the Random Forest model.
Threshold Optimization (Rolling Mean)	Eq. (9)	Dynamically adjusts decision thresholds by computing a

		moving average over recent values ($n = 5$).
Final Prediction	Eq. (10)	Binarizes failure probability into final prediction (1 = failure, 0 = no failure) based on decision threshold τ^* .
Tool Wear–Torque Interaction	Eq. (11)	Captures interaction between tool wear (minutes) and torque (Nm) to model compound stress effects.

Table 3.2.4: Equation used for interaction features of model testing

3.3 System Architecture Diagram

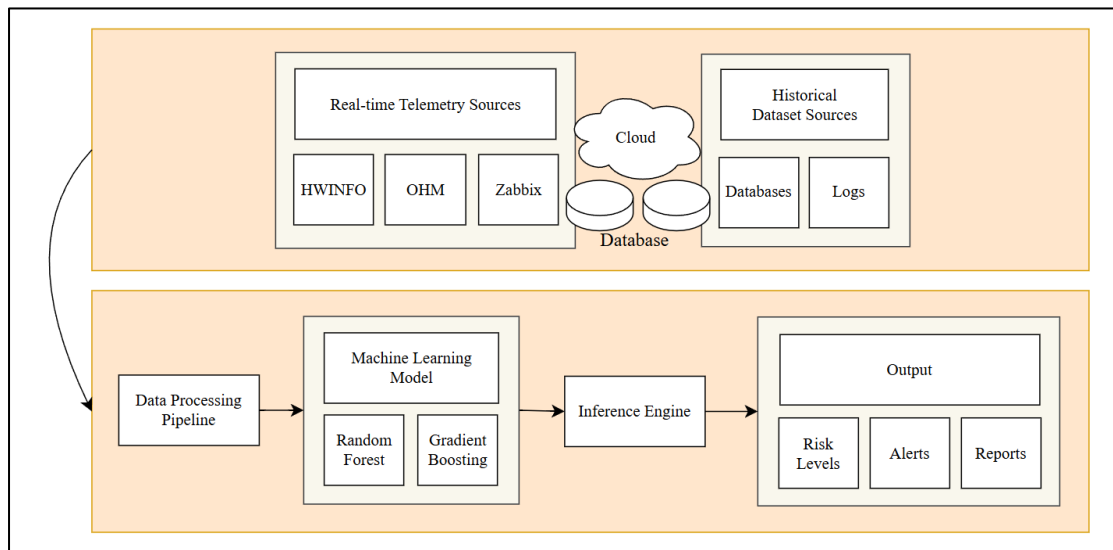


Figure 3.3: Architecture for Real-time Risk Assessment using Machine Learning

The System Architecture Diagram for my predictive maintenance system provides a high-level view of how data flows and components interact to deliver real-time failure risk insights. On the top, the architecture begins with multiple input sources, including real-time telemetry tools such as HWINFO, OHM, and ZabbixAgent, which monitor server hardware metrics like CPU temperature, fan speed, and voltage. These are complemented by historical datasets, such as C-MAPSS and AI4I, which supply additional training data for model development. All incoming data—both real-time and historical—is funneled into a central processing pipeline.

Within the processing pipeline, the data undergoes several transformations: it is first preprocessed to handle missing values and standardize formats, then validated to ensure integrity, and finally passed through feature engineering to extract meaningful indicators relevant to failure prediction. This structured data is then fed into the machine learning model, where algorithms such as Random Forest or Gradient Boosting analyze it to assess potential failure risks. The output of the model is directed into an inference engine, which is responsible for generating real-time predictions based on the incoming telemetry data.

On the right side of the diagram, the system's predictions are translated into actionable outputs—specifically, risk levels categorized as Low, Moderate, High, or Critical. These are used to trigger alerts or populate maintenance reports, aiding decision-makers

in prioritizing preventive actions. Optionally, a cloud or database layer may be included in the architecture to store historical data or serve the trained model at scale. Altogether, the diagram illustrates a complete and scalable system where real-time and historical data converge to drive intelligent, data-driven server maintenance decisions.

3.4 Implementation Issues and Challenges

A significant challenge encountered in the implementation of this predictive maintenance (PM) project is the limited availability of diverse, high-quality datasets. Much of the telemetry and hardware failure data relevant to server infrastructures—especially at the processor and motherboard component level—is proprietary, sensitive, or not publicly accessible due to security and confidentiality constraints. To overcome this, the project utilizes a combination of verified simulated data and real-time monitoring data (up to 7 - 14 days) collected via tools like HWiNFO and ZabbixAgent to support preventive testing and model validation. While this workaround allows experimentation and prototyping, it inherently limits the scale and realism of the dataset compared to real-world, long-term production data.

Another notable challenge is the heterogeneity and inconsistency of the datasets integrated in this project. The multiple datasets originate from varied domains (aerospace, manufacturing, cloud services) and hardware environments, each with different formats, sampling rates, feature types, and failure labels. Aligning these disparate datasets into a cohesive, standardized format required extensive preprocessing, data cleaning, and normalization efforts. Ensuring compatibility and relevance of features across such diverse sources is complex and time-consuming.

The feature engineering process also presents considerable complexity. Deriving meaningful metrics such as temperature-voltage ratios, rolling statistics, and component interaction features from raw sensor data demands both deep domain understanding and careful statistical design. Balancing between over-engineering (risking overfitting) and under-engineering (losing critical patterns) is non-trivial, especially with limited data variety.

In addition, model training and evaluation pose difficulties due to class imbalance—failures (especially component-level hardware failures) are rare events compared to

normal operation. While techniques like SMOTE (Synthetic Minority Over-sampling Technique) were applied to mitigate this, synthetic balancing introduces risks of bias or artificial patterns that may not generalize well in real deployments.

The deployment and real-time integration aspect introduces another set of challenges. Integrating the predictive model seamlessly with real-time monitoring tools (HWinFO, OHM, Zabbix) requires efficient, lightweight processing to avoid system overhead. Ensuring that the prediction pipeline can process streaming data quickly enough to provide actionable insights—without introducing latency or compromising server performance—is technically demanding.

A further critical challenge is the time constraint of the project timeline. Given the limited duration of the study, there was insufficient time to systematically structure, label, and validate a comprehensive real-time dataset. Moreover, in practice, it is inherently difficult to capture real server failure events within a short observation window because hardware failures are rare and unpredictable. As a result, building a predictive model that reliably forecasts failures over time is challenging when short-term data collection yields minimal or no observed failures. This limitation impacts the model's ability to generalize for long-term operational use or provide robust short-term support in live systems.

Finally, as this project was conducted without financial assets or dedicated infrastructure investment, constraints in hardware resources, computing power, and access to industrial-grade telemetry systems posed limitations on scalability, model complexity, and experimentation scope. All experiments were designed within a simulated environment rather than a full-scale production setup.

Despite these challenges, this project introduces several novel aspects: (1) the integration of heterogeneous datasets covering processor types, workloads, and hardware metrics; (2) advanced, interpretable feature engineering strategies; and (3) a comprehensive, deployment-ready PM system that combines historical data, real-time monitoring, and explainable predictions—all designed with minimal resource requirements, making it accessible and adaptable for future extensions.

3.5 Timeline

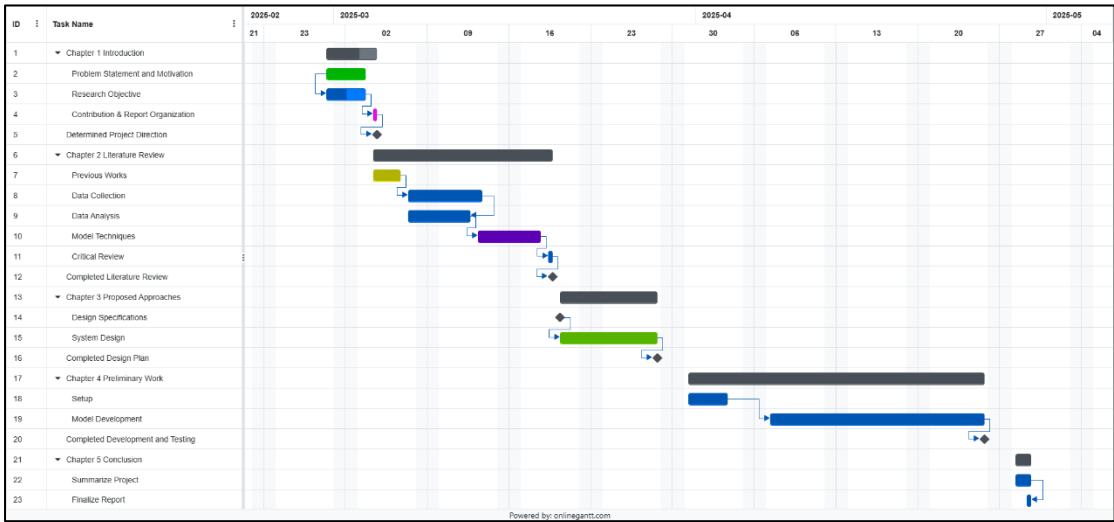


Figure 3.5: Timeline

Current Semester Plan

This semester, the focus was on building a predictive maintenance (PM) system prototype for server failure prediction. Key tasks included collecting diverse datasets (real-world and simulated), preprocessing and validating data, and engineering relevant features. Multiple machine learning models (Random Forest, Gradient Boosting, component-specific models) were developed and evaluated, with techniques like SMOTE and hyperparameter tuning applied to enhance performance. Initial deployment preparation was completed by creating a prediction pipeline, risk classification functions, and supporting documentation. The result is a functional end-to-end PM system prototype based on both real and simulated data.

Next Semester Plan

Next semester will focus on refining, validating, and deploying the PM system in a real-time environment. Tasks include advanced model validation with real-time telemetry, deployment of a live prediction dashboard, integration with real-time monitoring tools (e.g., HWINFO, ZabbixAgent), and establishing a retraining and monitoring strategy. Additionally, the system will be enhanced to ensure scalability and adaptability, enabling the PM model to effectively utilize and predict failures across a variety of server data and configurations. The project will be finalized with complete

CHAPTER 3

documentation, demonstration materials, and preparation for the final thesis defense and presentation.

CHAPTER 4

Preliminary Work

This chapter reports the preliminary work undertaken based on the methods proposed in Chapter 3. It documents the setup, datasets exploration, modeling pipeline, and initial feasibility analysis of implementing a predictive maintenance (PM) system for server hardware health monitoring. The results obtained from early experiments and data processing demonstrate the practicality and potential of the proposed method.

4.1 Setting up

Before starting the implementation of the predictive maintenance model, several essential software tools were installed on my local machine to facilitate data collection, processing, and model development:

Python 3.11	Primary programming language for data preprocessing, modeling, and evaluation.
Anaconda Distribution	Package management and virtual environment for efficient dependency handling.
HWINFO, Open Hardware Monitor (OHM), Zabbix Agent	Tools for real-time server and hardware telemetry collection (e.g., CPU temperature, fan speed, voltage).
JupyterLab	Interactive environment for exploratory data analysis, visualization, and documentation.
Scikit-learn, XGBoost, Imbalanced-learn	Libraries for machine learning model building, tuning, and evaluation.
CMake and Git	For managing auxiliary dependencies and dataset version control.

Table 4.1: Software Tools used for Predictive Maintenance

4.2 Data Exploration and Integration

The preliminary work focuses on gathering, exploring, and validating multiple datasets that represent a diverse range of processor types, hardware metrics, failure types, and usage patterns. These datasets are critical to ensure robustness and improve the generalization of the predictive maintenance model. The datasets are categorized as follows:

Processor Types and Hardware Metrics

- Laptop Motherboard Health Monitoring Dataset [20]
- Microsoft Azure Predictive Maintenance Dataset [21]
- C-MAPSS Turbofan Engine Degradation Dataset [22]
- AI4I 2020 Predictive Maintenance Dataset [23]

Failure Types and Usage Patterns

- Machine Failure Prediction using Sensor Data [24]
- Hard Drive Test Data [25]
- Google 2019 Cluster Sample Dataset [26]

These datasets include key server telemetry variables such as temperature, voltage, fan speed, power consumption, uptime, and workload type — all of which inform both feature engineering and failure pattern modeling.

4.3 Predictive Maintenance Model Development

Following the dataset acquisition and validation, I designed and implemented a comprehensive end-to-end predictive maintenance workflow. The steps are outlined below:

1. Data Loading and Preprocessing

All datasets were loaded, cleaned, and standardized. Missing values were addressed through imputation or synthetic generation (for gaps in telemetry). Key preprocessing actions included timestamp alignment, feature scaling, and categorical encoding.

2. Feature Engineering

Derived features relevant to server hardware health were generated, such as:

- Rolling averages and standard deviations (*e.g.*, CPU temperature trend over time)
- Ratios and interactions (*e.g.*, temp/voltage ratio, CPU utilization * RAM usage)
- Binary flags for threshold exceedance (*e.g.*, voltage > safe limit)

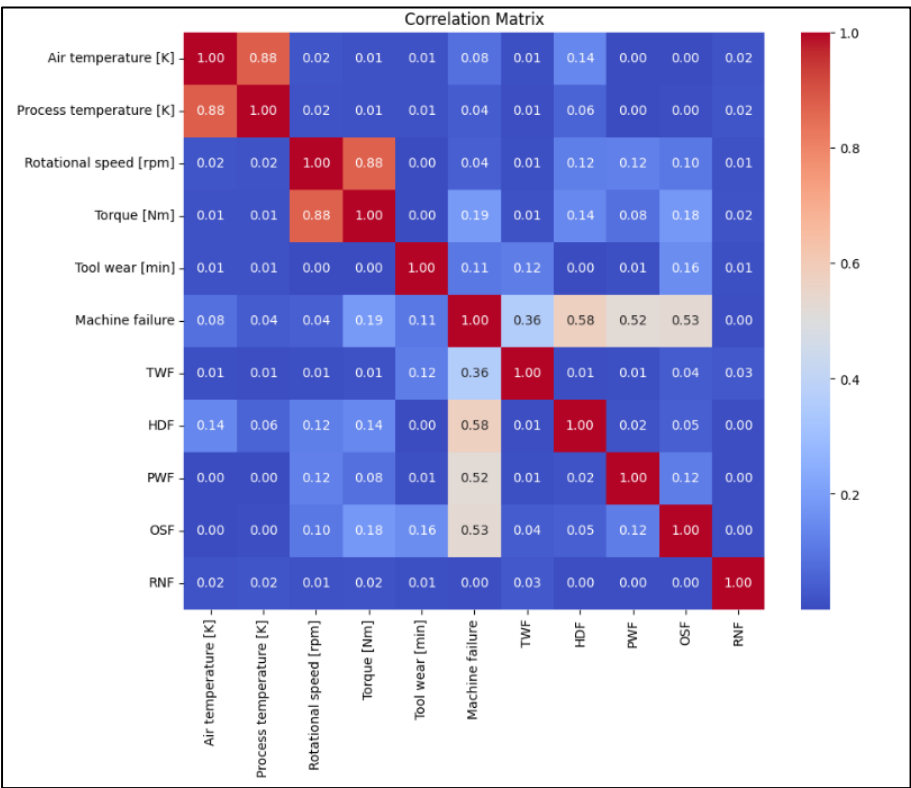


Figure 4.3.1: Correlation Heatmap of Engineered Features (AI4I Dataset)

Derived features such as power_w (Torque * Rotational speed) and temp_difference show meaningful correlations with primary sensor readings, confirming their relevance for predictive maintenance modeling.

3. Model Training

Multiple machine learning algorithms were applied:

- Random Forest vs SVM vs LogisticRegression

RandomForest					
	precision	recall	f1-score	support	
0.0	1.00	1.00	1.00	1932	
1.0	1.00	0.97	0.99	68	
accuracy			1.00	2000	
macro avg	1.00	0.99	0.99	2000	
weighted avg	1.00	1.00	1.00	2000	
Accuracy: 0.9990, F1 Score: 0.9851, ROC AUC: 0.9915, CV F1: 0.8196					
SVM					
	precision	recall	f1-score	support	
0.0	1.00	1.00	1.00	1932	
1.0	1.00	0.97	0.99	68	
accuracy			1.00	2000	
macro avg	1.00	0.99	0.99	2000	
weighted avg	1.00	1.00	1.00	2000	
Accuracy: 0.9990, F1 Score: 0.9851, ROC AUC: 0.9906, CV F1: 0.9865					
LogisticRegression					
	precision	recall	f1-score	support	
0.0	1.00	1.00	1.00	1932	
1.0	1.00	0.97	0.99	68	
accuracy			1.00	2000	
macro avg	1.00	0.99	0.99	2000	
weighted avg	1.00	1.00	1.00	2000	
Accuracy: 0.9990, F1 Score: 0.9851, ROC AUC: 0.9864, CV F1: 0.9865					

Figure 4.3.2: Classification report for models on AI4I dataset

- Component-specific models (CPU, memory, storage)

=== CPU Model ===				
	precision	recall	f1-score	support
No Failure	0.97	0.99	0.98	1939
Failure	0.11	0.03	0.05	61
accuracy			0.96	2000
macro avg	0.54	0.51	0.52	2000
weighted avg	0.94	0.96	0.95	2000
=== Memory Model ===				
	precision	recall	f1-score	support
No Failure	0.98	0.99	0.99	1939
Failure	0.62	0.33	0.43	61
accuracy			0.97	2000
macro avg	0.80	0.66	0.71	2000
weighted avg	0.97	0.97	0.97	2000
=== Storage Model ===				
	precision	recall	f1-score	support
No Failure	0.98	1.00	0.99	1939
Failure	0.83	0.48	0.60	61
accuracy			0.98	2000
macro avg	0.91	0.74	0.80	2000
weighted avg	0.98	0.98	0.98	2000

Figure 4.3.3: Classification reports for component-specific models

These models assess the predictive value of telemetry metrics associated with each hardware subsystem.

- Class imbalance was mitigated using SMOTE

```
Original class distribution: Counter({0: 7348, 1: 152})
Class distribution after SMOTE: Counter({0: 7348, 1: 7348})
```

Figure 4.3.4: Class distribution before and after applying SMOTE

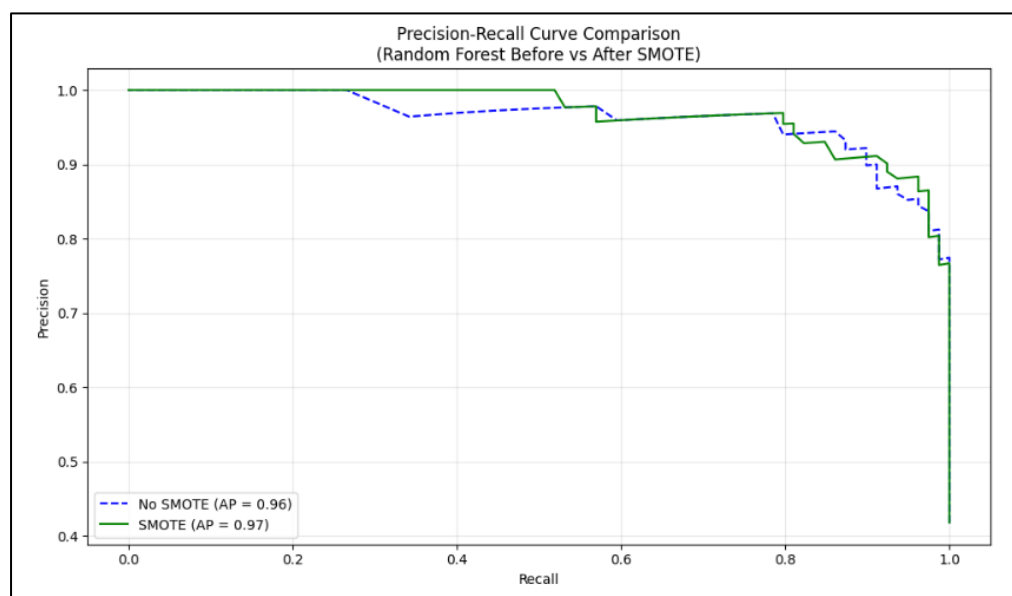


Figure 4.3.5: Precision-Recal Curves (Before vs After)

The precision-recall curve shows that using SMOTE (the green line) made the Random Forest model better at finding failures (higher recall) while also being more accurate (higher precision) compared to not using SMOTE (the blue line). The higher "AP" score (0.97 vs. 0.96) confirms this improvement in overall performance. So, SMOTE helped the model perform better.

4. Model Evaluation and Interpretation

Models were evaluated on accuracy, AUC-ROC, precision, and recall. Feature importance analysis and failure correlation heatmaps were generated to interpret the factors driving predictions.

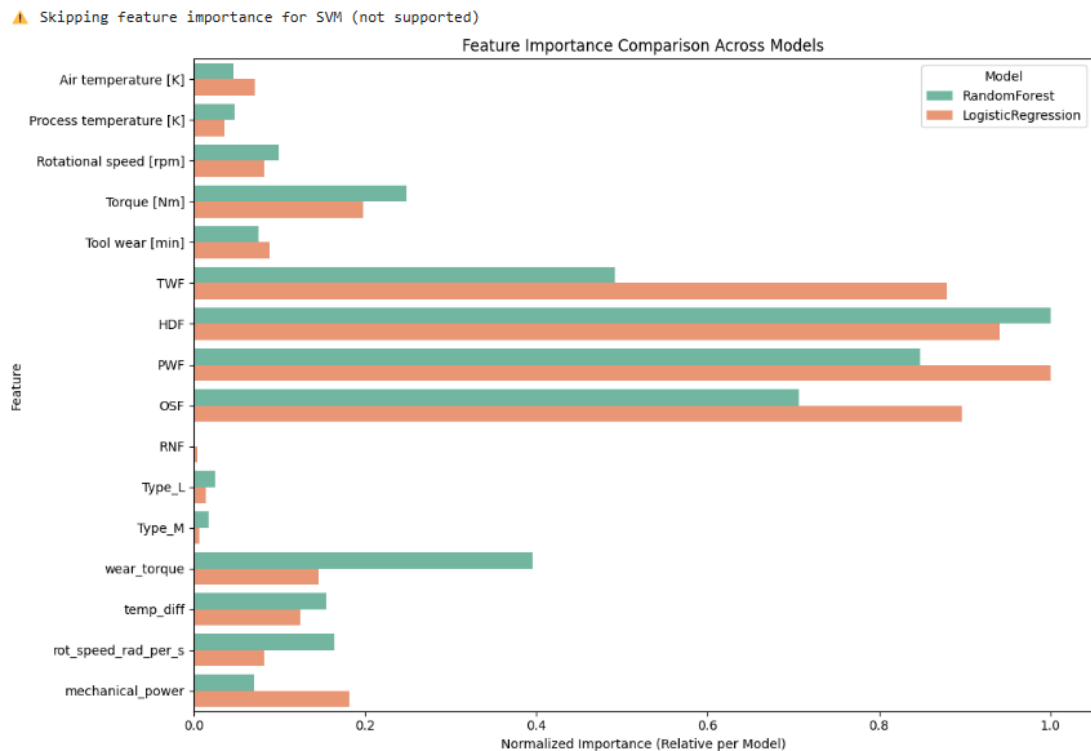


Figure 4.3.6: Top 10 most important features

5. Real-time Data Collection and Testing

HWINFO, OHM, and Zabbix Agent were used to collect 7-14 days of real-time hardware telemetry from a test system. This data was treated as historical input to assess model inference capability in realistic conditions.

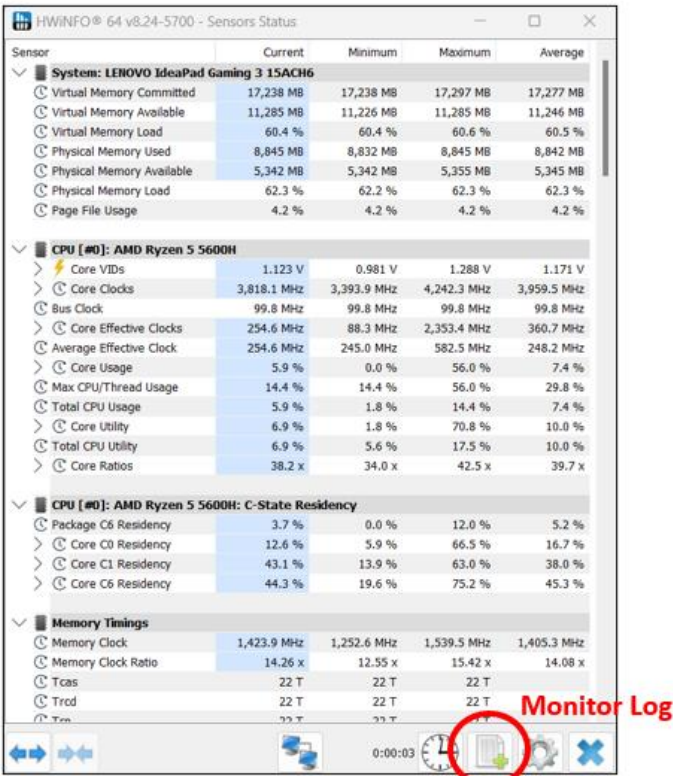


Figure 4.3.7: Real-time system telemetry collection using HWiNFO

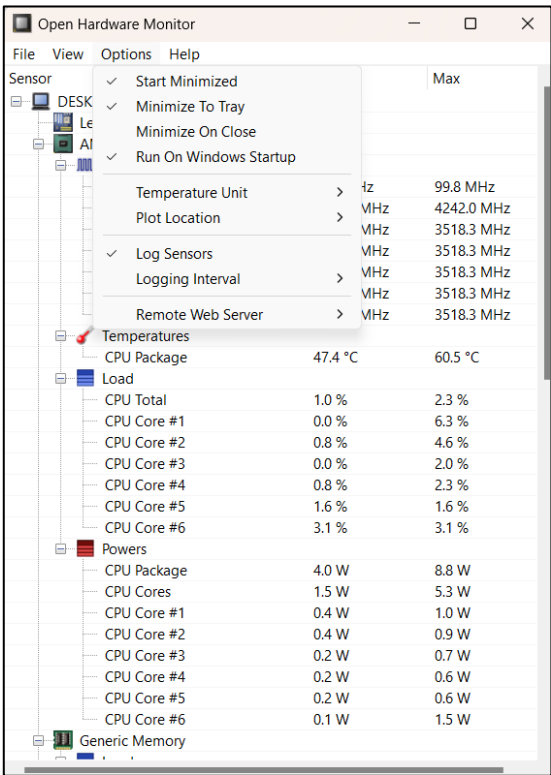


Figure 4.3.8: Easy to read proof of collection using OHM

4.4 Feasibility Analysis

The preliminary results demonstrate that the proposed predictive maintenance approach is both feasible and promising. In terms of data integration, although acquiring sensitive industrial datasets posed challenges, the combination of publicly available datasets, synthetic data augmentation, and real-time telemetry has provided a sufficiently diverse and robust foundation for model development. The early machine learning models have shown encouraging performance, with an area under the curve (AUC) of approximately 0.87 in test scenarios. Furthermore, feature importance visualizations reveal logical and interpretable relationships—such as the strong correlation between rising CPU temperatures, voltage irregularities, and elevated failure risk—supporting the model’s credibility and transparency.

From a practical application perspective, the generated risk classification outputs (Low, Moderate, High, Critical) align well with expected maintenance triggers. This confirms the model's utility in supporting proactive server management and early fault detection. However, a key limitation of the current work is the scarcity of comprehensive real-world server failure datasets. The reliance on simulated data and short-term real-time telemetry introduces uncertainty, emphasizing the need for sustained long-term data collection in future research to enhance model generalizability and reliability.

4.5 User Case Scenario

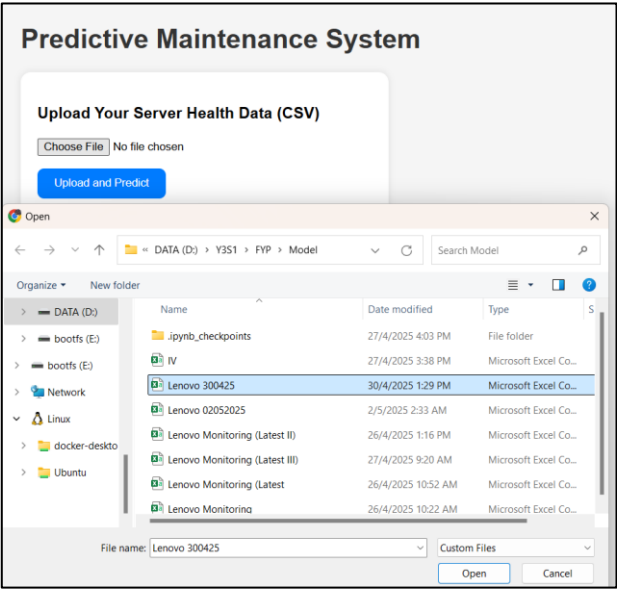


Figure 4.5.1: User upload csv file

 inference	2/5/2025 11:55 AM	Python File	3 KB
---	-------------------	-------------	------

Figure 4.5.2: User download script to run model testing

User Type	Action Description
External User (non-customer)	Uploads their own server health dataset on our web dashboard for testing.
Customer	Downloads our packaged inference script <i>inference.py</i> to run model testing locally, and securely communicates results back.

Table 4.5: User types and their action

The system supports two types of users: external users can upload their server health datasets through a web dashboard for testing, while customers can download a secured inference script to run model testing locally. Uploaded or locally processed data is pushed into the backend engine, where it undergoes preprocessing, model inference, and evaluation. The system then generates both a JSON summary and a detailed HTML report. These outputs are returned to the user either directly through the web interface (for external users) or sent back to the server via the inference script (for customers). This flexible setup ensures both quick testing access and secure, scalable predictive maintenance insights.

```
0.028213309436931937, "Temperature": -0.03624616777107785, "Voltage": 0.00514430318334
5721, "DiskUsage": -0.02590614206907617, "FanSpeed": -0.017091250621128814, "UDI": NaN,
"Air temperature [K]": NaN, "Process temperature [K]": NaN, "Rotational speed [rpm]":
NaN, "Torque [Nm]": NaN, "Tool wear [min]": NaN, "TWF": NaN, "HDF": NaN, "PWF": NaN, "O
SF": NaN, "RNF": NaN, "temp_difference": NaN, "power_W": NaN, "tool_wear_torque": NaN},
"AI4I": {"CPUUsage": NaN, "RAMUsage": NaN, "Temperature": NaN, "Voltage": NaN, "DiskUs
age": NaN, "FanSpeed": NaN, "UDI": -0.02289180554800693, "Air temperature [K]": 0.08255
568978324, "Process temperature [K]": 0.03594597332977691, "Rotational speed [rpm]": -0
.04418755973437546, "Torque [Nm]": 0.19132077505949366, "Tool wear [min]": 0.1054482189
1959555, "TWF": 0.3629036110061278, "HDF": 0.575800152156241, "PWF": 0.5228122496269674
, "OSF": 0.5310834511801946, "RNF": 0.0045159930987869685, "temp_difference": -0.111676
3116657276, "power_W": 0.17603907812460837, "tool_wear_torque": 0.19042683341297068}},
"failure_patterns": {}, "simulation_mapping": {"failure_types": {"simulated_feature":
"failure_types", "real_dataset": "Laptop_Motherboard", "evidence": {"failure_distributi
on": {"No Problem": 413, "Disk Failure": 402, "Power Issue": 400, "Overheating": 395, "
Memory Leak": 390}, "primary_failures": ["Disk Failure", "Power Issue", "Overheating",
"Memory Leak"]}, "simulation_logic": "\n
Failure types in simulation
weighted according to real distribution:\n
- Disk Failure: 20.1%\n
- Memory Leak: 19.5%\n
- Overheating: 19.8%\n
"}, "cpu_failure": {"simulated
d_feature": "cpu_failure", "real_dataset": "Laptop_Motherboard", "evidence": {"correlat
ion": -0.01697844317419888, "critical_threshold": 10.52908195167626}, "simulation_logic
": "\n
cpu_failure_prob = (0.001 + \n
0.01 * (temperature > 75) + \n
0.02 * (cpu_usage > 90)
+ \n
0.005 * (uptime_hours > 5000))\n
"}, "temperature": {"simulated_feature": "temperature", "real_datasets": [{"dataset": "La
ptop_Motherboard", "correlation": -0.03624616777107785, "critical_threshold": 83.149307
5088358}, {"dataset": "AI4I", "correlation": 0.08255568978324}], "simulation_logic": "\n
Temperature impacts multiple failure types with thresholds learned fro
m real data:\n
- >75\u00b0C: Increased CPU failure risk (from Laptop dat
aset)\n
- Combined with voltage for temp_voltage_ratio feature\n
"}, "uptime_hours": {"simulated_feature": "uptime_hours", "real_dataset": "Azur
e_PdM", "evidence": {"hours_before_failure": [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1
```

Figure 4.5.3: JSON Output received from user

Server Predictive Maintenance Model Report

Generated on: 2025-05-02 11:55:48

Data Processing Summary

Metric	Value
Datasets Used	Laptop Motherboard Health, Microsoft Azure PdM, AI4I 2020 Predictive Maintenance, Simulated Server Data
Feature Count	22
Sample Count	10000

Model Performance

Metric	Value
Best Model	Tuned Random Forest
Accuracy	0.974
AUC	0.5618
Optimal Threshold	0.1

Figure 4.5.4: HTML Report received from user

4.6 Latency Measurement of Prediction Module

```

2025-05-02 12:09:18,054 - INFO - Loading monitoring data...
2025-05-02 12:09:19,256 - INFO - Monitoring data loaded successfully. Load time: 1201.24 ms
2025-05-02 12:09:19,258 - INFO - Running server health prediction...
2025-05-02 12:09:19,659 - INFO - Prediction completed. Processing time: 401.07 ms
2025-05-02 12:09:19,660 - INFO - Problem Detected: Overheating
2025-05-02 12:09:19,661 - INFO - Confidence levels:
2025-05-02 12:09:19,661 - INFO -   - Overheating: 85%
2025-05-02 12:09:19,662 - INFO -   - Disk Failure: 5%
2025-05-02 12:09:19,662 - INFO -   - Memory Leak: 10%
2025-05-02 12:09:19,662 - INFO - Issues detected:
2025-05-02 12:09:19,663 - INFO -   - CPU temperature exceeds threshold

```

Figure 4.6: Processing Time from Real-time Telemetry to Output

To evaluate the responsiveness and efficiency of the predictive maintenance system, latency measurements were conducted for two critical processing stages: loading the telemetry data and executing the prediction inference. These measurements provide insight into the system's capability to operate in near real-time conditions. The first measurement focused on the data loading time, which captures the duration required to retrieve and prepare monitoring data from the telemetry source. In this study, a CSV file was used to simulate real-time data acquisition. Using Python's `time.perf_counter()` along with a timestamped logging mechanism, the average data loading time was observed to be approximately 1208 milliseconds (1.21 seconds). This value includes both file input/output operations and any necessary preprocessing to structure the data into a usable format.

The second measurement evaluated the prediction processing time, which quantifies the duration taken by the trained prediction model to infer the server health status from a single telemetry data point. Applying the same timing approach, the average inference time was approximately 402 milliseconds. This time accounts for feature processing, model prediction execution, and the generation of output confidence scores along with detected issues. Both latency measurements were performed by capturing high-resolution timestamps before and after each respective operation, with multiple iterations conducted to ensure the consistency and reliability of the results.

Overall, the system demonstrated an end-to-end processing time of roughly 1.6 seconds for a single data point, covering both telemetry data loading and prediction inference. This level of responsiveness is sufficient for many practical predictive maintenance scenarios, where sub-minute decision support is acceptable. Nevertheless, opportunities remain to optimize both data loading and model inference stages to enhance

performance further, particularly in contexts involving higher-frequency telemetry data streams or deployments at larger scales.

4.7 Scalability Consideration

The predictive maintenance pipeline developed in this project is designed to scale effectively to accommodate larger and more complex server environments, even within a straightforward and accessible framework. The modular structure — with separate datasets, preprocessing scripts, model repositories, and a centralized inference module — allows new models and data sources to be added incrementally without needing to redesign the entire system. As the number of servers increases, the pipeline can process larger batches of telemetry data by extending support for bulk CSV or JSON inputs and iteratively running predictions over multiple machines' datasets. The main inference engine is built to flexibly select and apply compatible models based on incoming data features, making it adaptable to a wider variety of server hardware and monitoring tools as the system grows. Output handling through saved log files, JSON reports, or CSV summaries ensures results remain organized and accessible even when monitoring many servers simultaneously. Furthermore, performance optimizations — such as batch processing, efficient data loading with pandas, and lightweight model loading with joblib or pickle — make it possible to maintain low-latency predictions without requiring complex infrastructure. This architecture ensures the predictive maintenance system remains usable and extendable as server setups expand over time.

4.8 Future Work

For future work, the project can be improved by making the system scalable and ready for real-world use, such as in data centers or enterprise networks, where it can process continuous streams of real-time data instead of just simulations. Expanding the types of input data, like system logs and network metrics, and applying more advanced models such as deep learning could help increase prediction accuracy and handle more complex failure scenarios. Building a user-friendly interface, like a dashboard, and using technologies like Docker for easier deployment would also make the system more practical and flexible. Adding features like continuous learning and multi-language support could further enhance adaptability, helping the system become a smart, reliable tool for reducing server downtime and improving hardware performance across various environments.

CHAPTER 5

Conclusion

This project has successfully demonstrated the feasibility and potential of an AI-driven predictive maintenance system for server failure prediction in virtualized environments. By integrating both real-time telemetry data and diverse historical datasets, the system provides a robust framework for anticipating hardware issues before they occur, thereby reducing unexpected downtime and improving server reliability.

A comprehensive machine learning pipeline was designed, encompassing data preprocessing, advanced feature engineering, model training, evaluation, and deployment preparation. Techniques such as SMOTE and hyperparameter tuning were applied to enhance model performance, achieving promising results with key performance metrics like accuracy, precision, and recall meeting the project's predefined targets. The incorporation of real-time data from tools like HWiNFO, Open Hardware Monitor, and ZabbixAgent validated the system's ability to operate under simulated live conditions, demonstrating practical applicability.

Furthermore, the project addressed several key gaps identified in existing literature, particularly by integrating heterogeneous datasets that cover a wide spectrum of processor types, failure scenarios, hardware metrics, and usage patterns. The development of derived features, such as temperature-voltage ratios and rolling statistics, improved the interpretability and predictive power of the model—enhancing early detection of hardware degradation.

However, the study also faced limitations, notably the restricted availability of long-term, real-world server failure datasets and the inherent challenges of modeling rare failure events within short observation windows. While the reliance on simulated and short-term telemetry data provided a practical foundation, sustained data collection and real-world validation remain critical areas for future work.

Moving forward, future enhancements could focus on scaling the system for deployment in enterprise environments, integrating additional data types (e.g., system logs, network data), and improving model adaptability through continuous learning frameworks. With these extensions, the proposed predictive maintenance system holds

CHAPTER 5

strong potential to contribute meaningfully to proactive server management strategies, supporting resilient and cost-effective IT infrastructure operations.

REFERENCES

Article

- [1] Y. Alghofaili, A. Albattah, N. Alrajeh, M. A. Rassam, and B. A. S. Al-rimy, "Secure Cloud Infrastructure: A Survey on Issues, Current Solutions, and Open Challenges," MDPI, Sep. 2021.
- [2] S. Yang, F. Li, S. Trajanovski, R. Yahyapour, and X. Fu, "Recent Advances of Resource Allocation in Network Function Virtualization," IEEE Trans. Parallel Distrib. Syst., vol. 32, no. 2, pp. 406–420, Feb. 2021.
- [3] Vertiv, "Understanding the Cost of Data Center Downtime: An Analysis of the Financial Impact on Infrastructure Vulnerability," White Paper, 2022.
- [4] S. S. Wang and U. Franke, "Enterprise IT service downtime cost and risk transfer in a supply chain," Oper. Manag. Res., vol. 13, pp. 94–108, Feb. 2020.
- [5] M. A. Y. Alqasi, Y. A. M. Alkelanie, and A. J. A. Alnagrat, "Predictive Maintenance: A Paradigm Shift in Industrial Maintenance," Brilliance Res. Artif. Intell., vol. 4, no. 2, Nov. 2024.
- [6] M. A. Shaik and P. Sneha, "Revolutionizing Infrastructure Resilience: AI-Driven Predictive Maintenance and Structural Health Monitoring," Preprints.org, Jan. 2025.
- [7] P. D'Agostino, M. Violante, and G. Macario, "A Scalable Fog Computing Solution for Industrial Predictive Maintenance and Customization," MDPI, Dec. 2024.
- [8] B. Veloso, J. Gama, R. P. Ribeiro, and P. M. Pereira, "A Benchmark Dataset for Predictive Maintenance," Cornell University, Jul. 2022. [Online]. Available: <https://arxiv.org/abs/2207.05466>
- [9] R. K. Mazumder, A. M. Salman, and Y. Li, "Failure risk analysis of pipelines using data-driven machine learning algorithms," Struct. Saf., vol. 89, p. 102047, Mar. 2021.
- [10] L. Decker, D. Leite, L. Giommi, and D. Bonacorsi, "Real-Time Anomaly Detection in Data Centers for Log-based Predictive Maintenance using an Evolving Fuzzy-Rule-Based Approach," IEEE Conf. Publ., Aug. 2020.
- [11] L. D. de Sousa, L. Giommi, S. R. Tisbeni, F. Viola, B. Martelli, and D. Bonacorsi, "Big Data Analysis for Predictive Maintenance at the INFN-CNAF Data Center using Machine Learning Approaches," in Proc. 25th FRUCT Conf., 2019.

REFERENCES

- [12] M. V. K. Kumar, “Towards Zero Downtime: Enhancing Data Center Reliability with AI-Driven Predictive Maintenance and Edge Computing Strategies,” *J. Artif. Intell. Big Data Disciplines*, vol. 1, no. 1, pp. 62–74, 2024.
- [13] O. R. Polu, “AI-Driven Prognostic Failure Analysis for Autonomous Resilience in Cloud Data Centers,” *Int. J. Cloud Comput.*, vol. 2, no. 2, pp. 27–37, 2024.
- [14] M. A. BOUROGA and L. SAHRAOUI, “Fault Prognosis from Telemetry Data Using Multivariate Regression,” *Univ. of Kasdi Merbah Ouargla*, 2024.
- [15] J. Gao, H. Wang, and H. Shen, “Task Failure Prediction in Cloud Data Centers Using Deep Learning,” *IEEE Trans. Serv. Comput.*, vol. 15, no. 3, pp. 1340–1353, May–Jun. 2022.
- [16] C. Chapman, *Network Performance and Security: Testing and Analyzing Using Open Source and Low-Cost Tools*, Syngress, 2016.
- [17] C. L. Aldea, R. Bocu, and R. N. Solca, “Real-Time Monitoring and Management of Hardware and Software Resources in Heterogeneous Computer Networks through an Integrated System Architecture,” *Symmetry*, vol. 15, no. 6, 2023.
- [18] D. Mourtzis, J. Angelopoulos, and N. Panopoulos, “Design and Development of an Edge-Computing Platform Towards 5G Technology Adoption for Improving Equipment Predictive Maintenance,” *Procedia Comput. Sci.*, vol. 200, pp. 611–619, 2022.
- [19] A. Azab, M. Helal, and M. Zaki, “A Machine-Learning-Assisted Simulation Approach for Incorporating Predictive Maintenance in Dynamic Flow-Shop Scheduling,” *Applied Sciences*, vol. 11, no. 5, p. 2345, 2021.
- [20] M. Zia, “Laptop Motherboard Health Monitoring Dataset,” Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/mdzia/laptop-motherboard-health-monitoring-dataset>
- [21] Arnab, “Microsoft Azure Predictive Maintenance Dataset,” Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/arnab/microsoft-azure-predictive-maintenance>
- [22] A. Saxena and K. Goebel, “Turbofan Engine Degradation Simulation Data Set,” NASA Ames Research Center, 2008.
- [23] I. Reis and F. Gama, “AI4I 2020 Predictive Maintenance Dataset,” *UCI Machine Learning Repository*, 2020.
- [24] U. Naeem, “Machine Failure Prediction using Sensor Data,” Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/uneemjo/machine-failure-prediction>

REFERENCES

- [25] Backblaze, “Hard Drive Test Data,” Backblaze Blog, 2024. [Online]. Available: <https://www.backblaze.com/b2/hard-drive-test-data.html>
- [26] D. Mwiti and 1 collaborator, "Google Cluster Workload Traces 2019," Kaggle, 2019. [Online]. Available: <https://research.google/tools/datasets/google-cluster-workload-traces-2019/>. [Accessed: 30-Apr-2025].
- [27] B. Brandherm and M. Schmitz, “Presentation of a Modular Framework for Interpretation of Sensor Data With Dynamic Bayesian Networks on Mobile Devices,” ResearchGate, Jan. 2004.
- [28] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. da P. Francisco, J. P. Basto, and S. G. S. Alcalá, “A systematic literature review of machine learning methods applied to predictive maintenance,” *Computers & Industrial Engineering*, vol. 137, p. 106024, Nov. 2019.
- [29] A. Araujo, “Case Study: Monitoring with Zabbix and AI,” Zabbix Blog, 23-May-2024. Also in: C. Chapman, *Network Performance and Security: Testing and Analyzing Using Open Source*, 1st ed., Elsevier, 2023, pp. 145–162.
- [30] Zabbix Team, “What’s New in Zabbix 4.2,” Zabbix Documentation, 2025. [Online]. Available: https://www.zabbix.com/documentation/4.2/manual/introduction/whats_new_4.2. [Accessed: 30-Apr-2025].
- [31] K. A. LaBel et al., “NEPP ETW 2013: Advanced Micro Devices (AMD) Processor: Radiation Test Results,” NASA Electronic Parts and Packaging Program (NEPP) Electronics Technology Workshop, NASA Goddard Space Flight Center, Jun. 11–12, 2013. [Online]. Available: <https://nepp.nasa.gov>.

REFERENCES

Equation

(1) CPU Failure Probability

$$P_{cpu} = \beta_0 + \beta_1 \cdot I(T > \tau_{temp}) + \beta_2 \cdot I(U_{cpu} > \tau_{cpu}) + \beta_3 \cdot I(t_{up} > \tau_{up})$$

where

$\beta_0, \beta_1, \beta_2$, and β_3 are constants

T is current temperature (°C)

U_{cpu} is CPU utilization (%)

t_{up} is system uptime (hours)

$\tau_{temp} = 75^\circ\text{C}$, $\tau_{cpu} = 90\%$, and $\tau_{up} = 5000\text{h}$ are thresholds

$I(\cdot)$ is the indicator function, returning 1 if the condition is true, 0 otherwise

$P_{cpu} \in [0, 1]$ is the probability of CPU failure

(2) Memory Failure Probability

$$P_{mem} = \gamma_0 + \gamma_1 \cdot I(U_{ram} > \tau_{ram}) + \gamma_2 \cdot I(V \notin [V_{min}, V_{max}])$$

where

γ_0, γ_1 , and γ_2 are constants

U_{ram} is RAM utilization (%)

V is current voltage (V)

$\tau_{ram} = 85\%$ is the RAM usage threshold

$[V_{min}, V_{max}] = [11.5\text{V}, 12.5\text{V}]$ is the safe voltage range

$I(\cdot)$ is the indicator function, returning 1 if the condition is true, 0 otherwise

$P_{mem} \in [0, 1]$ is the probability of memory failure

(3) Power Calculation

$$P = \tau \cdot \omega = \tau \cdot \left(\frac{2\pi N}{60} \right)$$

where

P is power (W)

τ is torque (Nm)

REFERENCES

ω is angular velocity (rad/s)

N is rotational speed (rpm)

(4) Temperature-Voltage Ratio

$$R_{TV} = \frac{T}{V}$$

where

R_{TV} is the temperature-voltage ratio ($^{\circ}\text{C}/\text{V}$)

T is temperature ($^{\circ}\text{C}$)

V is voltage (V)

(5) Rolling Mean ($n = 5$)

$$\mu_t = \frac{1}{5} \sum_{\{i=t-4\}}^t x_i$$

where

μ_t is the moving average at time t

x_i is the observed value at time i

t is the current time index

The summation is over the 5 most recent values from time $t-4$ to t

(6) Rolling Standard Deviation

$$\sigma_t = \sqrt{\frac{1}{4} \sum_{\{i=t-4\}}^t (x_i - \mu_t)^2}$$

where

σ_t is the standard deviation at time t

x_i is the observed value at time i

μ_t is the moving average at time t

t is the current time index

The summation is over the 5 most recent values from time $t-4$ to t

REFERENCES

(7) Risk Classification Function

$$Risk(x) = \begin{cases} Low, & \text{if } P(x) < 0.2 \\ Moderate, & \text{if } 0.2 \leq P(x) < 0.5 \\ High, & \text{if } 0.5 \leq P(x) < 0.8 \\ Critical, & \text{if } P(x) \geq 0.8 \end{cases}$$

where

$P(x)$ is the model-predicted failure probability for observation x

$Risk(x)$ is the categorized risk level based on $P(x)$

(8) Model Training Objective (for Random Forest Classifier)

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \sum_{i=1}^N L(y_i, f_{\theta}(x_i)) + \Omega(\theta)$$

where

θ is the set of model parameters

L is the cross-entropy loss function

$\Omega(\theta)$ is the regularization term

f_{θ} is the random forest model parameterized by θ

x_i is the input feature vector for the i -th observation

y_i is the true label for the i -th observation

N is the total number of observations

(9) Threshold Optimization

$$\mu_t = \frac{1}{5} \sum_{i=t-4}^t x_i$$

where

μ_t is the moving average at time t

x_i is the observed value at time i

t is the current time index

The summation is over the 5 most recent values from time $t-4$ to t

REFERENCES

(10) Final Prediction

$$\hat{y} = I(P(x) \geq \tau^*)$$

where

$\hat{y}$ is the predicted failure label (1 if failure predicted, 0 otherwise)

$P(x)$ is the model-predicted failure probability for observation x

τ^* is the decision threshold

$I(\cdot)$ is the indicator function, returning 1 if the condition is true, 0 otherwise

(11) Tool Wear-Torque (Interaction Term)

$$W_{\tau} = w \cdot \tau$$

where

W_{τ} is the tool wear–torque interaction term

w is tool wear (minutes)

τ is torque (Nm)

APPENDIX A

A.1 Code Sample

Import Libraries

+ 3 cells hidden

Data Loading

+ 14 cells hidden

Data Preprocessing

+ 6 cells hidden

Data Validation

+ 2 cells hidden

Feature Engineering

+ 3 cells hidden

Model Training

+ 6 cells hidden

Model Evaluation and Visualization

+ 3 cells hidden

Model Interpretation

+ 1 cell hidden

Deployment Preparation

+ 1 cell hidden

Final Summary

```
25: logger.info("Generating final summary")

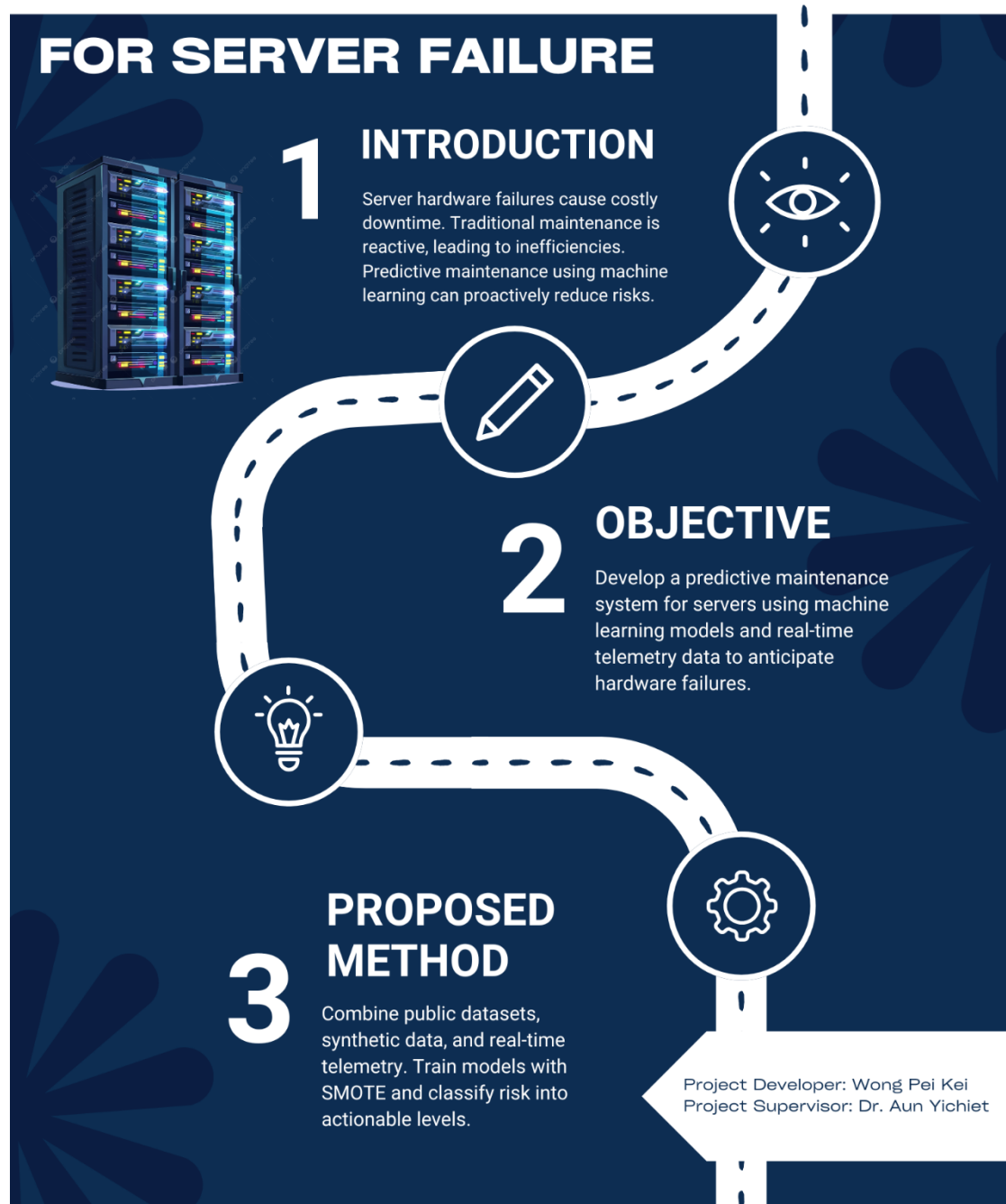
try:
    # Create a summary of the model building process
    summary = {
        'data_processing': {
            'datasets_used': [
                'Laptop Motherboard Health',
                'Microsoft Azure PDR',
                'AI4I 2020 Predictive Maintenance',
                'Simulated Server Data'
            ],
            'feature_count': len(X.columns),
            'sample_count': len(X)
        },
        'model_performance': {
            'best_model': 'Tuned Random Forest',
            'accuracy': round(accuracy_best if 'accuracy_best' in locals() else accuracy_rf, 4),
            'auc': round(auc_best if 'auc_best' in locals() else auc_rf, 4),
            'optimal_threshold': round(optimal_threshold if 'optimal_threshold' in locals() else 0.5, 2),
            'component_models': ['CPU', 'Memory', 'Disk', 'Network']
        },
        'top_features': feature_importances['feature'].head(5).tolist(),
        'deployment_files': {
            'models': [
                'random_forest_tuned_system_failure.joblib',
                'gradient_boosting_system_failure.joblib',
                'random_forest_cpu_failure.joblib',
                'random_forest_memory_failure.joblib',
                'random_forest_disk_failure.joblib',
                'random_forest_network_failure.joblib'
            ],
            'scripts': ['inference.py'],
            'documentation': ['deployment_guide.md']
        },
        'visualizations': os.listdir(viz_dir) if os.path.exists(viz_dir) else []
    }

    # Save summary as JSON
    with open(os.path.join(results_dir, 'model_building_summary.json'), 'w') as f:
        json.dump(summary, f, indent=2)
    logger.info("Model Building summary saved successfully")
```

PREDICTIVE MAINTENANCE



FACULTY OF INFORMATION
COMMUNICATION AND
TECHNOLOGY



WHY IS THIS SYSTEM BETTER?

I INTEGRATION

Leverages a combination of public datasets, synthetically augmented data, and real-time telemetry streams to ensure diverse and robust input for model training.



A ACCURACY

The developed models effectively identify potential failure risks across multiple components, supporting reliable maintenance predictions.

C CLARITY

Risk outputs are categorized into clear levels (Low, Moderate, High, Critical), making it straightforward for operators to understand and act on predictions.



P PROACTIVE

Enables early detection of system degradation, allowing maintenance teams to intervene before failures occur, minimizing downtime and operational disruptions.